

DeepSeek-V4: 迈向高效的百万 Token 上下文智能

DeepSeek-AI
research@deepseek.com

摘要

我们发布了 DeepSeek-V4 系列的预览版本，其中包括两个强大的专家混合（MoE）语言模型——拥有 1.6T 参数（激活 49B）的 DeepSeek-V4-Pro，以及拥有 284B 参数（激活 13B）的 DeepSeek-V4-Flash——二者都支持一百万 Token 的上下文长度。DeepSeek-V4 系列在架构与优化方面引入了若干关键升级：（1）结合压缩稀疏注意力（CSA）与重度压缩注意力（HCA）的混合注意力架构，以提升长上下文效率；（2）增强传统残差连接的流形约束超连接（mHC）；（3）以及用于更快收敛和更高训练稳定性的 Muon 优化器。我们在超过 32T 的多样化高质量 Token 上对两个模型进行了预训练，随后通过完整的后训练流水线进一步释放并增强其能力。DeepSeek-V4-Pro-Max 作为 DeepSeek-V4-Pro 的最大推理强度模式，重新定义了开源模型的最新水平，在核心任务上超越其前代。同时，DeepSeek-V4 系列在长上下文场景中也具有极高效率。在一百万 Token 上下文设定下，与 DeepSeek-V3.2 相比，DeepSeek-V4-Pro 仅需其单 Token 推理 FLOPs 的 27% 和 KV 缓存的 10%。这使我们能够常规性地支持一百万 Token 的上下文，从而让长时程任务以及进一步的测试时扩展更具可行性。模型检查点可在 <https://huggingface.co/collections/deepseek-ai/deepseek-v4> 获取。

making long-horizon tasks and further test-time scaling more feasible. The model checkpoints are available at <https://huggingface.co/collections/deepseek-ai/deepseek-v4>.

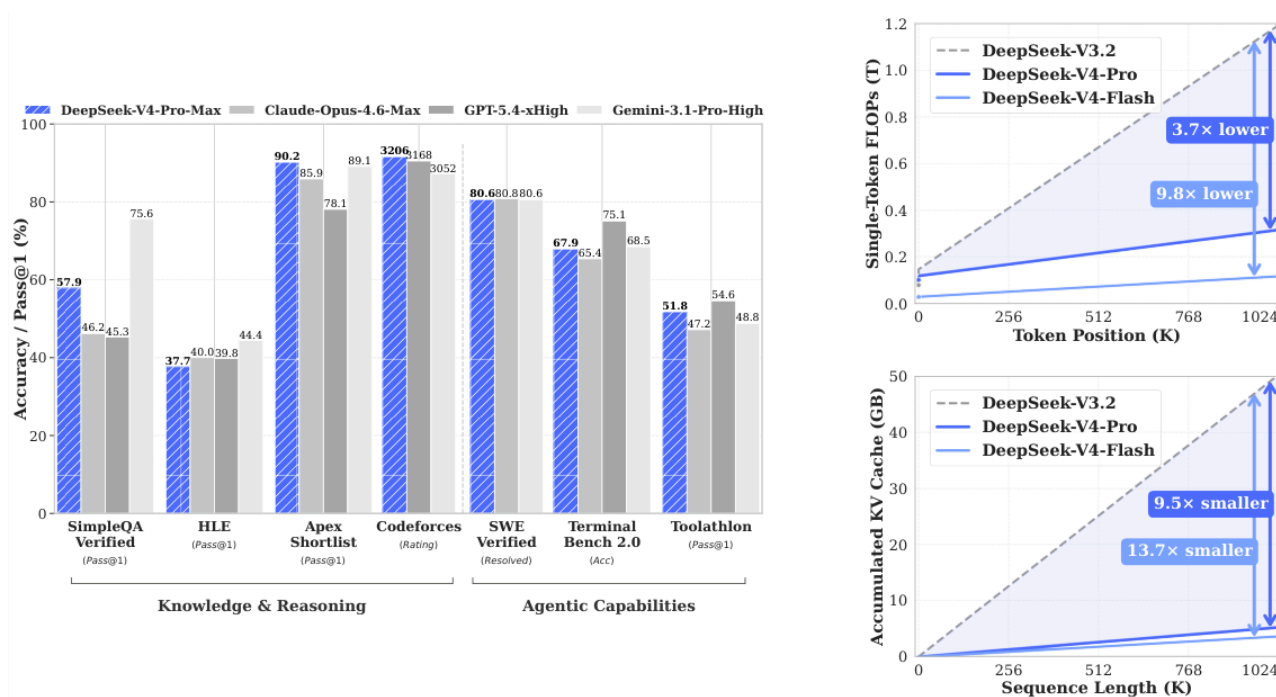


图 1 | 左：DeepSeek-V4-Pro-Max 及其对比模型的基准性能。右：DeepSeek-V4 系列与 DeepSeek-V3.2 的推理 FLOPs 和 KV 缓存大小。

目录

1	引言	4
2	架构	6
2.1	继承自 DeepSeek-V3 的设计	7
2.2	流形约束超连接	7
2.3	结合 CSA 与 HCA 的混合注意力	9
2.3.1	压缩稀疏注意力	9
2.3.2	重度压缩注意力	11
2.3.3	其他细节	12
2.3.4	效率讨论	13
2.4	Muon 优化器	14
3	通用基础设施	15
3.1	专家并行中的细粒度通信—计算重叠	15
3.2	使用 TileLang 的灵活高效内核开发	16
3.3	高性能、批大小无关且确定性的内核库	18
3.4	FP4 感知量化训练	19
3.5	训练框架	20
3.5.1	Muon 的高效实现	20
3.5.2	mHC 的低成本且内存高效实现	21
3.5.3	用于长上下文注意力的上下文并行	21
3.5.4	用于灵活激活检查点的扩展自动微分	21
3.6	推理框架	22
3.6.1	KV 缓存结构与管理	22
3.6.2	磁盘上的 KV 缓存存储	23
4	预训练	24
4.1	数据构建	24
4.2	预训练设置	25
4.2.1	模型设置	25
4.2.2	训练设置	25
4.2.3	缓解训练不稳定性	26
4.3	评测	27
4.3.1	评测基准	27
4.3.2	评测结果	28
5	后训练	29
5.1	后训练流水线	29
5.1.1	专家化训练	29

5.1.2	On-Policy 蒸馏	32
5.2	RL 与 OPD 基础设施	34
5.2.1	FP4 量化集成	34
5.2.2	面向全词表 OPD 的高效教师调度	34
5.2.3	可抢占且具容错能力的 Rollout 服务	34
5.2.4	面向百万 Token 上下文的可扩展 RL 框架	35
5.2.5	用于智能体 AI 的沙箱基础设施	35
5.3	标准基准评测	36
5.3.1	评测设置	36
5.3.2	评测结果	38
5.4	真实世界任务中的表现	41
5.4.1	中文写作	41
5.4.2	搜索	42
5.4.3	白领任务	42
5.4.4	代码智能体	44
6	结论、局限性与未来方向	44
A	作者列表与致谢	54
B	评测细节	55

1. 引言

推理模型 (DeepSeek-AI, 2025; OpenAI, 2024c) 的出现, 确立了一种新的测试时扩展范式, 为大语言模型 (LLM) 带来了显著的性能提升。然而, 这一扩展范式从根本上受到标准注意力机制 (Vaswani et al., 2017) 二次计算复杂度的限制, 这为超长上下文和推理过程带来了难以承受的瓶颈。与此同时, 从复杂的智能体工作流到大规模跨文档分析, 长时程场景与任务的兴起, 也使得对超长上下文的高效支持成为未来进展的关键。尽管近期的开源工作 (Bai et al., 2025a; DeepSeek-AI, 2024; MiniMax, 2025; Qwen, 2025) 推动了通用能力的发展, 但在处理超长序列时这种核心架构低效性仍然是一个重要障碍, 限制了测试时扩展进一步带来的收益, 也阻碍了对长时程场景和任务的进一步探索。

为了打破超长上下文中的效率壁垒, 我们开发了 DeepSeek-V4 系列, 包括拥有 1.6T 参数 (激活 49B) 的 DeepSeek-V4-Pro 预览版, 以及拥有 284B 参数 (激活 13B) 的 DeepSeek-V4-Flash 预览版。通过架构创新, DeepSeek-V4 系列在处理超长序列时实现了计算效率的巨大跃升。这一突破使得高效支持一百万 Token 上下文长度成为可能, 开启了下一代大语言模型的百万长度上下文时代。我们认为, 高效处理超长序列的能力解锁了测试时扩展的下一前沿, 为深入研究长时程任务铺平了道路, 并为探索在线学习等未来范式奠定了必要基础。

与 DeepSeek-V3 架构 (DeepSeek-AI, 2024) 相比, DeepSeek-V4 系列保留了 DeepSeekMoE 框架 (Dai et al., 2024) 和多 Token 预测 (MTP) 策略, 同时在架构与优化方面引入了若干关键创新。为提升长上下文效率, 我们设计了一种结合压缩稀疏注意力 (CSA) 与重度压缩注意力 (HCA) 的混合注意力机制。CSA 会沿序列维度压缩 KV 缓存, 然后执行 DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025); 而 HCA 则对 KV 缓存施加更激进的压缩, 但保留稠密注意力。为增强建模能力, 我们引入了流形约束超连接 (mHC) (Xie et al., 2026), 用于升级传统残差连接。此外, 我们还将 Muon (Jordan et al., 2024; Liu et al., 2025) 优化器引入 DeepSeek-V4 系列的训练中, 从而带来更快的收敛速度和更好的训练稳定性。

为了支持 DeepSeek-V4 系列的高效训练与推理, 并提高开发生产力, 我们引入了多项基础设施优化。首先, 我们为 MoE 模块设计并实现了单个融合内核, 使计算、通信和内存访问能够完全重叠。其次, 我们采用 TileLang (Wang et al., 2026) 这一领域专用语言 (DSL), 以兼顾开发效率与运行时效率。第三, 我们提供了高效、与批大小无关且确定性的内核库, 以确保训练和推理过程中的按位可复现性。第四, 我们为 MoE 专家权重以及 indexer 的 QK 路径引入 FP4 感知量化训练, 以降低内存占用与计算开销。第五, 在训练框架方面, 我们通过张量级检查点扩展自动求导框架, 以实现细粒度重计算控制; 同时, 我们还通过面向 Muon 优化器的混合 ZeRO 策

略、借助重计算与融合内核实现的低成本 mHC、以及用于管理压缩注意力的两阶段上下文并行来提升训练效率。最后，在推理框架方面，我们设计了带有磁盘存储策略的异构 KV 缓存结构，以支持高效的共享前缀复用。

原文第 5 页

通过采用混合 CSA 和 HCA，并结合计算与存储精度优化，DeepSeek-V4 系列相较于 DeepSeek-V3.2 实现了显著更低的推理 FLOPs 和大幅缩减的 KV 缓存规模，尤其是在长上下文设置下更为明显。图 1 右侧展示了 DeepSeek-V3.2 与 DeepSeek-V4 系列估算的单 Token 推理 FLOPs 和累计 KV 缓存大小。在 1M Token 上下文场景中，即便是激活参数更多的 DeepSeek-V4-Pro，其单 Token FLOPs（以等效 FP8 FLOPs 计）也仅为 DeepSeek-V3.2 的 27%，KV 缓存大小仅为其 10%。此外，激活参数更少的 DeepSeek-V4-Flash 将效率进一步推进：在 1M Token 上下文设置下，其单 Token FLOPs 仅为 DeepSeek-V3.2 的 10%，KV 缓存大小仅为其 7%。另外，DeepSeek-V4 系列的路由专家参数采用 FP4 精度。虽然在现有硬件上， $FP4 \times FP8$ 运算的峰值 FLOPs 目前与 $FP8 \times FP8$ 相同，但理论上在未来硬件上它们可实现高出 1/3 的效率，从而进一步提升 DeepSeek-V4 系列的效率。

在预训练阶段，我们分别用 32T Token 训练 DeepSeek-V4-Flash，用 33T Token 训练 DeepSeek-V4-Pro。经过预训练后，这两个模型都能够原生且高效地支持 1M 长度上下文。在我们的内部评测中，DeepSeek-V4-Flash-Base 凭借更高参数效率的设计，已经在大多数基准上超越 DeepSeek-V3.2-Base。DeepSeek-V4-Pro-Base 则进一步扩大了这一优势，在 DeepSeek 基座模型中树立了新的性能标准，在推理、代码、长上下文和世界知识任务上实现了全面领先。

DeepSeek-V4 系列的后训练流水线采用两阶段范式：先独立培养领域专长专家，再通过 on-policy distillation (Lu and Lab, 2025) 进行统一模型整合。起初，对于每个目标领域——如数学、编码、智能体和指令遵循——都会独立训练一个专家模型。基础模型首先在高质量、领域特定数据上进行监督微调 (SFT)，以建立基础能力。随后，使用群体相对策略优化 (GRPO) (DeepSeek-AI, 2025) 进行强化学习 (RL)，在针对特定成功标准定制的奖励模型引导下，进一步将模型优化为符合领域目标的行为。这一阶段产出了一组多样化的专门专家，每个专家都在其对应领域表现出色。最后，为整合这些不同专长，我们通过 on-policy distillation 训练一个统一模型，其中统一模型作为学生，通过教师模型学习优化反向 KL 损失。

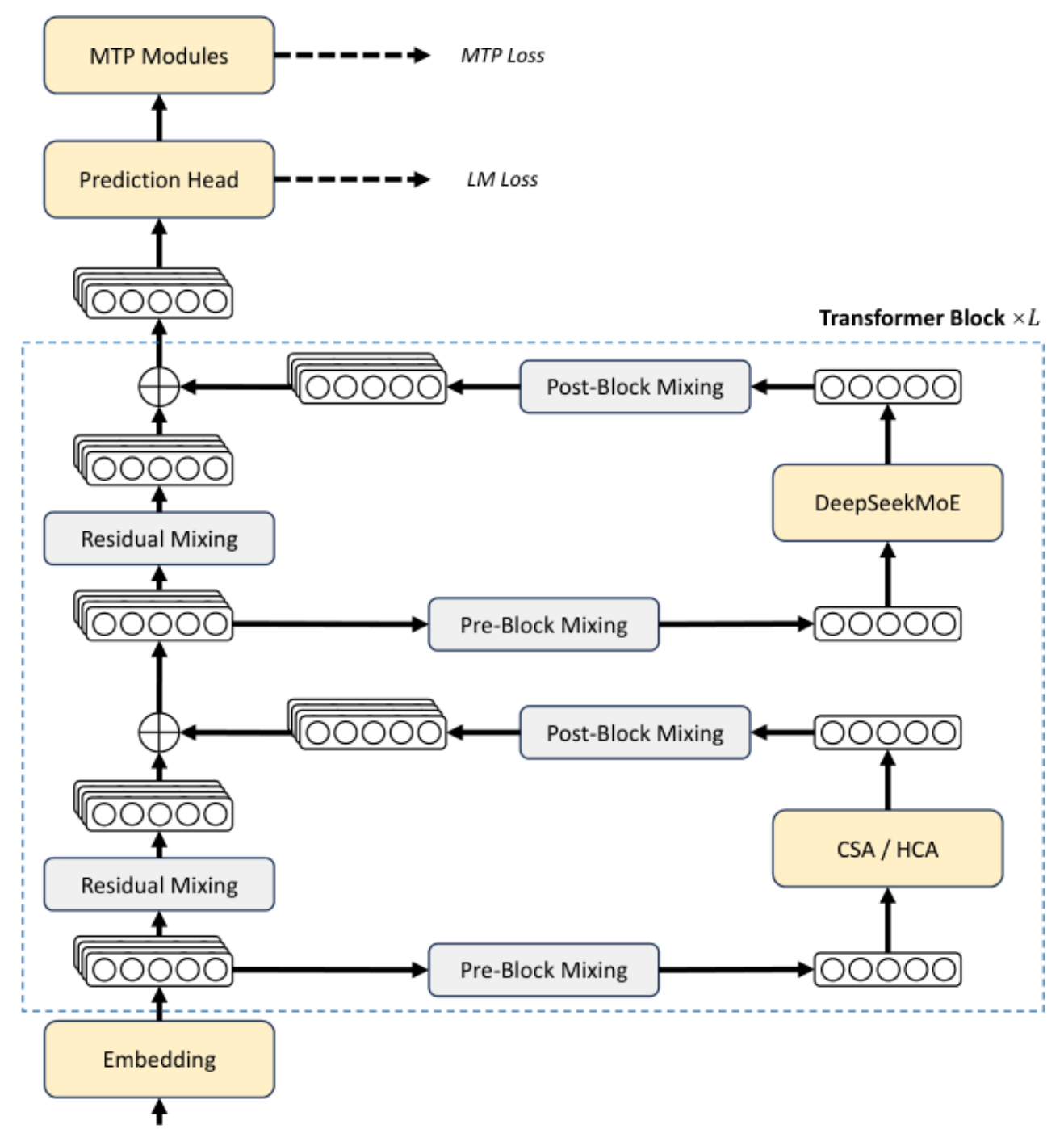
核心评测结果概览

- 知识：在广泛世界知识评测中，DeepSeek-V4-Pro-Max 作为 DeepSeek-V4-Pro 的最大推理强度模式，在 SimpleQA (OpenAI, 2024d) 和 Chinese-SimpleQA (He et al., 2024) 基准上显著优于领先的开源模型。在教育知识方面——通过 MMLU-Pro (Wang et al., 2024b)、HLE (Phan et al., 2025) 和 GPQA (Rein et al., 2023) 评测——DeepSeek-V4-Pro-Max 相比其

开源对手也略占优势。尽管在这些知识类评测中仍稍逊于领先的专有模型 Gemini-3.1-Pro，DeepSeek-V4-Pro-Max 已显著缩小与其之间的差距。

- 推理：随着推理 Token 数量的扩展，DeepSeek-V4-Pro-Max 在标准推理基准上相较 GPT-5.2 和 Gemini-3.0-Pro 展现出更优性能。不过，其表现仍略逊于 GPT-5.4 和 Gemini-3.1-Pro，这表明其发展进程相比最前沿模型大约落后 3 到 6 个月。此外，DeepSeek-V4-Flash-Max 也实现了与

原文第 6 页



输入 Token

与 GPT-5.2 和 Gemini-3.0-Pro 相当的性能，确立了其作为复杂推理任务高性价比架构的地位。

- 智能体：在公开基准上，DeepSeek-V4-Pro-Max 与 Kimi-K2.6 和 GLM-5.1 等领先开源模型处于同一水平，但略逊于前沿闭源模型。在我们的内部评测中，DeepSeek-V4-Pro-Max 优于 Claude Sonnet 4.5，并接近 Opus 4.5 的水平。
- 长上下文：DeepSeek-V4-Pro-Max 在拥有 100 万 Token 上下文窗口的合成与真实使用场景中都展现出强劲结果，在学术基准上甚至超过 Gemini-3.1-Pro。
- DeepSeek-V4-Pro 与 DeepSeek-V4-Flash 对比：由于参数规模更小，DeepSeek-V4-Flash-Max 在知识类评测中的表现较低。然而，在分配更大思考预算时，它在推理任务上取得了可比结果。在智能体评测中，虽然 DeepSeek-V4-Flash-Max 在若干基准上与 DeepSeek-V4-Pro-Max 表现相当，但在更复杂、更高难度的任务上仍落后于更大的版本。

2. 架构

总体而言，DeepSeek-V4 系列保留了 Transformer (Vaswani et al., 2017) 架构和多 Token 预测 (MTP) 模块 (DeepSeek-AI, 2024; Gloeckle et al., 2024)，同时相较 DeepSeek-V3 引入了若干关键升级：（1）首先，我们引入流形约束超连接 (mHC) (Xie et al., 2026) 以强化传统残差连接；

图 2 | DeepSeek-V4 系列的整体架构。我们在注意力层中使用混合 CSA (压缩稀疏注意力) 和 HCA (重度压缩注意力)，在前馈层中使用 DeepSeekMoE，并通过 mHC 强化传统残差连接。

原文第 7 页

（2）其次，我们设计了一种混合注意力架构，通过压缩稀疏注意力与重度压缩注意力大幅提升长上下文效率。（3）第三，我们采用 Muon (Jordan et al., 2024; Liu et al., 2025) 作为优化器。对于专家混合 (MoE) 组件，我们仍采用 DeepSeekMoE (Dai et al., 2024) 架构，仅相较 DeepSeek-V3 做了少量调整。多 Token 预测 (MTP) (DeepSeek-AI, 2024; Gloeckle et al., 2024; Li et al., 2024; Qi et al., 2020) 的配置与 DeepSeek-V3 完全一致。其他未特别说明的细节均沿用 DeepSeek-V3 (DeepSeek-AI, 2024) 中设定。图 2 展示了 DeepSeek-V4 的整体架构，具体细节如下所述。

2.1. 继承自 DeepSeek-V3 的设计

专家混合。与以往的 DeepSeek 系列模型 (DeepSeek-AI, 2024; DeepSeek-AI, 2024) 一样，DeepSeek-V4 系列也在前馈网络 (FFN) 中采用 DeepSeekMoE 范式 (Dai et al., 2024)，设置了细粒度路由专家和共享专家。与 DeepSeek-V3 不同的是，我们将用于计算亲和分数的激活函数由 Sigmoid($\cdot$) 改为 Sqrt(Softplus($\cdot$))。在负载均衡方面，我们同样采用无辅助损失策略 (DeepSeek-AI, 2024; Wang et al., 2024a)，并辅以轻量的按序列平衡损失，以防止单个序列内部出现极端失衡。对于 DeepSeek-V4，我们移除了对路由目标节点数量的约束，并

仔细重新设计了并行化策略，以维持训练效率。此外，与 DeepSeek-V3 相比，我们将最初若干个 Transformer 块中的稠密 FFN 层替换为使用 Hash 路由（Roller et al., 2021）的 MoE 层。Hash 路由策略会根据预定义的哈希函数以及输入 Token ID 来确定每个 Token 的目标专家。

多 Token 预测。与 DeepSeek-V3 一样，DeepSeek-V4 系列也设置了 MTP 模块和目标。鉴于 MTP 策略已在 DeepSeek-V3 中得到验证，我们在 DeepSeek-V4 系列中不作修改地沿用了相同策略。

2.2. 流形约束超连接

如图 2 所示，DeepSeek-V4 系列引入了流形约束超连接（mHC）（Xie et al., 2026），用于强化相邻 Transformer 块之间的传统残差连接。与朴素超连接（HC）（Zhu et al., 2025）相比，mHC 的核心思想是将残差映射约束在特定流形上，从而在保留模型表达能力的同时增强跨层信号传播的稳定性。本小节将简要介绍标准 HC，并说明我们如何设计 mHC 以实现稳定训练。

标准超连接。标准 HC 将残差流的宽度扩展为原来的 n_{hc} 倍。具体而言，残差流的形状从 R_d 扩展为 $R_{n_{hc} \times d}$ ，其中 d 是实际层输入的隐藏维度。设 $X_l = [x_{l,1}; \dots; x_{l,n_{hc}}]^T \in R_{n_{hc} \times d}$ 表示第 l 层之前的残差状态。HC 引入了三个线性映射：输入映射 $A_l \in R_{1 \times n_{hc}}$ 、残差变换 $B_l \in R_{n_{hc} \times n_{hc}}$ ，以及输出映射 $C_l \in R_{n_{hc} \times 1}$ 。随后，残差状态的更新可表述为：

$$X_{l+1} = B_l X_l + C_l \mathcal{F}_l(A_l X_l), \quad (1)$$

其中， $\mathcal{F}_l$ 表示第 l 层（例如一个 MoE 层），其输入和输出形状均为 R_d 。请注意，实际层输入 $A_l X_l \in R_d$ 同样是 d 维的，因此扩展后的残差

原文第 8 页

宽度不会影响内部层的设计。HC 将残差宽度与实际隐藏维度解耦，提供了一个额外的扩展轴，而且由于 n_{hc} 通常远小于隐藏维度 d ，因此计算开销很小。然而，尽管 HC 已展示出改善模型性能的潜力，我们发现当堆叠多层时，训练过程会频繁出现数值不稳定，从而阻碍 HC 的扩展。

流形约束残差映射。mHC 的核心创新在于将残差映射矩阵 B_l 约束到双随机矩阵流形（Birkhoff 多面体） $\mathcal{M}$ 上，从而增强跨层信号传播的稳定性：

$$B_l \in \mathcal{M} := \{M \in \mathbb{R}^{n \times n} \mid M \mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^T M = \mathbf{1}_n^T, M \geq 0\}. \quad (2)$$

这一约束确保映射矩阵的谱范数 $\|B_l\|_2$ 被 1 所界定，因此残差变换是非扩张的，这提高了前向传播和反向传播过程中的数值稳定性。此外，集合 $\mathcal{M}$ 在乘法下封闭，这保证了在深层堆叠 mHC

场景下的稳定性。除此之外，输入变换 A_l 和输出变换 C_l 也通过 Sigmoid 函数被约束为非负且有界，以避免信号抵消的风险。

动态参数化。三个线性映射的参数是动态生成的，可分解为动态（依赖输入）部分和静态（与输入无关）部分。给定输入 $X_l \in \mathbb{R}^{n_{hc} \times d}$ ，首先将其展平并归一化： $\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1 \times n_{hcd}}$ 。随后，我们遵循传统 HC 的方式生成不受约束的原始参数 $\tilde{A}_l \in \mathbb{R}^{1 \times n_{hc}}$ 、 $\tilde{B}_l \in \mathbb{R}^{n_{hc} \times n_{hc}}$ 以及 $\tilde{C}_l \in \mathbb{R}^{n_{hc} \times 1}$ ：

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}}, \quad (3)$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}}, \quad (4)$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, \quad (5)$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}}, (5)$$

其中， $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{hcd} \times n_{hc}}$ ， $W_l^{\text{res}} \in \mathbb{R}^{n_{hcd} \times n_{2hc}}$ ，是用于生成动态部分的可学习参数； $\text{Mat}(\cdot)$ 会将一个大小为 $1 \times n_{2hc}$ 的向量重塑为一个大小为 $n_{hc} \times n_{hc}$ 的矩阵；

$S_l^{\text{pre}} \in \mathbb{R}^{1 \times n_{hc}}$ 、 $S_l^{\text{post}} \in \mathbb{R}^{n_{hc} \times 1}$ ，以及 $S_l^{\text{res}} \in \mathbb{R}^{n_{hc} \times n_{hc}}$ 是可学习的静态偏置； α_l^{pre} 、 α_l^{res} 、 $\alpha_l^{\text{post}} \in \mathbb{R}$ 是可学习的门控因子，并初始化为较小的数值。

施加参数约束。在获得不受约束的原始参数 $\tilde{A}_l$ 、 $\tilde{B}_l$ 、 $\tilde{C}_l$ 之后，我们再对它们施加前述约束，以增强数值稳定性。具体来说，对于输入映射和输出映射，我们使用 Sigmoid 函数 $\sigma(\cdot)$ 来确保其非负性和有界性：

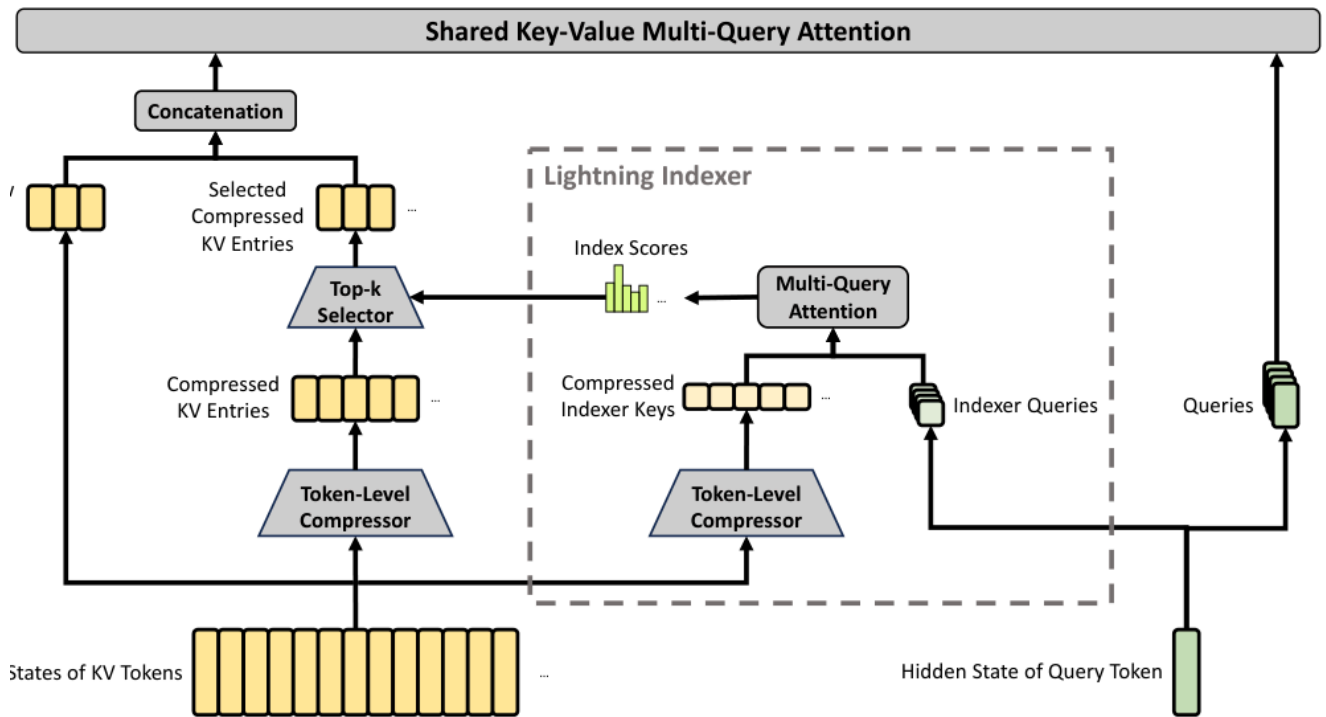
$$A_l = \sigma(\tilde{A}_l), \quad (6)$$

$$C_l = 2\sigma(\tilde{C}_l). \quad (7)$$

$$C_l = 2\sigma(\tilde{C}_l). (7)$$

对于残差映射 $\tilde{B}_l$ ，我们将其投影到双随机矩阵流形 $\mathcal{M}$ 上。具体通过 Sinkhorn-Knopp 算法实现：先对 $\tilde{B}_l$ 应用指数函数以保证正性，得到 $M(0) = \exp(\tilde{B}_l)$ ，再迭代执行列归一化与行归一化： $M(t) = \text{Tr}(\text{Tc}(M(t-1)))$, (8)

其中， Tr 和 Tc 分别表示行归一化与列归一化。该迭代会收敛到受约束的双随机矩阵 $B_l = M(t_{\text{max}})$ 。我们选择 $t_{\text{max}} = 20$ 作为一个实用取值。



滑动窗口KV 条目

m 倍，并随后应用 DeepSeek Sparse Attention 以进一步加速。此外，还会将少量滑动窗口 KV 条目与选中的压缩 KV 条目结合起来，以增强局部细粒度依赖建模。

2.3. 结合 CSA 与 HCA 的混合注意力

当上下文长度达到极端规模时，注意力机制会成为模型中的主要计算瓶颈。对于 DeepSeek-V4，我们设计了两种高效注意力架构——压缩稀疏注意力（CSA）和重度压缩注意力（HCA）——并采用二者交错的混合配置，从而在长文本场景下显著降低注意力的计算成本。CSA 同时结合压缩与稀疏注意力策略：它首先将每 m 个 Token 的键值（KV）缓存压缩为一个条目，然后应用 DeepSeek Sparse Attention（DSA）（DeepSeek-AI, 2025），使每个查询 Token 仅关注 k 个压缩后的 KV 条目。HCA 则以极致压缩为目标，将每 m' ($\gg m$) 个 Token 的 KV 缓存合并为一个条目。CSA 与 HCA 的混合架构显著提升了 DeepSeek-V4 系列的长上下文效率，使一百万 Token 上下文在实践中成为可能。本小节介绍我们混合注意力架构的核心技术；我们还提供了开源实现¹，以便无歧义地说明更多细节。

2.3.1. 压缩稀疏注意力

CSA 的核心架构如图 3 所示：它首先将每 m 个 Token 的 KV 缓存压缩为一个条目，然后应用 DeepSeek Sparse Attention 进行进一步加速。

压缩键值条目。设 $H \in R^{n \times d}$ 为一段输入隐藏状态序列，其中 n 是序列长度， d 是隐藏维度。CSA 首先计算两组 KV 条目 $C_a, C_b \in R^{n \times c}$ 及其对应的压缩权重 $Z_a, Z_b \in R^{n \times c}$ ，其中 c 是头

图 3 | CSA 的核心架构。它将 KV 条目的数量压缩到原来的 1

原文第 10 页

维度：

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV}, \quad (9)$$

$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ}, \quad (10)$$

$$Za = H \cdot WaZ, Zb = H \cdot WbZ, (10)$$

其中, $WaKV$ 、 $WbKV$ 、 WaZ 、 $WbZ \in \mathbb{R}^{d \times c}$ 是可训练参数。接下来, Ca 和 Cb 中每 m 个 KV 条目会根据对应的压缩权重以及可学习的位置偏置 Ba 、 $Bb \in \mathbb{R}^{m \times c}$ 被压缩为一个条目, 生成 $CComp \in \mathbb{R}^{nm \times c}$ 。每个压缩条目 $CComp_i \in \mathbb{R}^c$ 的计算方式为

$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]), \quad (11)$$

$$m(i+1)-1 \sum$$

$$\cdot \sum_{j=m(i-1)}^{m(i+1)-1} S_j^b \odot C_j^b, \quad (12)$$

$$CComp_i =$$

$$j=mi S_{aj} \odot C_{aj} +$$

$$j=m(i-1) S_{bj} \odot C_{bj}, (12)$$

其中, $\odot$ 表示 Hadamard 积; $\text{Softmax}_{\text{row}}(\cdot)$ 表示沿行维度进行 softmax 运算, 也就是在来自 Za 与 Zb 的总共 $2m$ 个元素上进行归一化。当 $i = 0$ 时, $Zb_{m(i-1):mi-1}$ 会用负无穷填充, 而 $Cb_{m(i-1):mi-1}$ 会用

零进行填充。需要注意的是, 每个 $CComp_i$ 都由 $2m$ 个 KV 条目得到, 但用于

$CComp_i$ 的 Cb 索引与用于 $CComp_{i-1}$ 的 Ca 索引存在重叠。因此, CSA 实际上将序列长度压缩到原来的 1

m 倍。

用于稀疏选择的 Lightning Indexer。在获得压缩后的 KV 条目 $cComp$ 之后，CSA 会应用 DSA 策略，为核心注意力选择 top-k 压缩 KV 条目。首先，CSA 使用与 $cComp$ 相同的压缩操作得到压缩后的 indexer key: $KIComp \in R^{nm \times cl}$ ，其中 cl 是 indexer 头维度。随后，对于查询 Token t ，我们以低秩方式生成 indexer query $\{q_{It,1}; q_{It,2}; \dots; q_{It,nlh}\}$:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (13)$$

$$[\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,nlh}^I] = \mathbf{q}_t^I = \mathbf{c}_t^Q \cdot W^{IUQ}, \quad (14)$$

$$[q_{It,1}; q_{It,2}; \dots; q_{It,nlh}] = q_{It} = c_{Qt} \cdot W^{IUQ}, (14)$$

其中， $\mathbf{h}_t \in R^d$ 是查询 Token t 的输入隐藏状态； $c_{Qt} \in R^{dc}$ 是查询的压缩潜变量； dc 表示查询压缩维度； nlh 表示 indexer 查询头的数量； $W^{DQ} \in R^{d \times dc}$ 和 $W^{IUQ} \in R^{dc \times clnlh}$ 分别是 indexer query 的下投影和上投影矩阵。接下来，查询 Token t 与其前方某个压缩块 s ($s < \text{Floor}(t)$

m)) 的索引分数 $I_{t,s} \in R$ 计算为

$$[w_{t,1}^I; w_{t,2}^I; \dots; w_{t,nlh}^I] = \mathbf{w}_t^I = \mathbf{h}_t \cdot W^w, \quad (15)$$

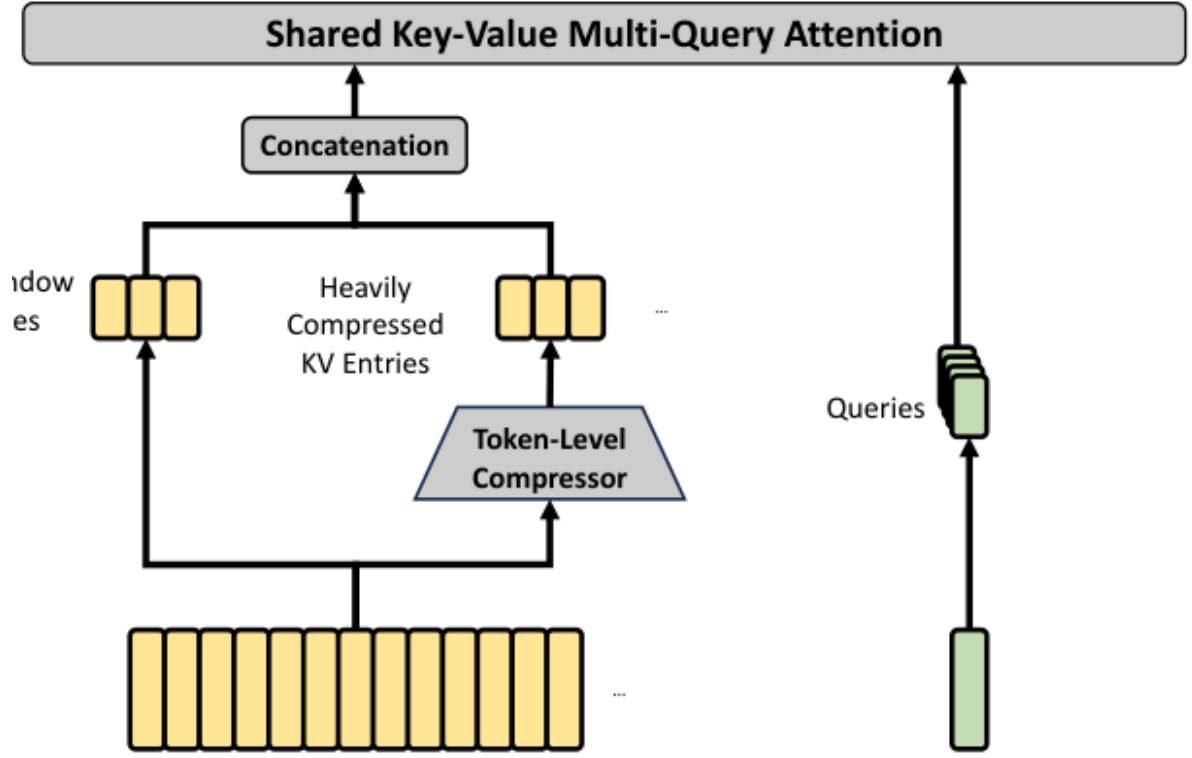
$$\sum_{h=1}^{nlh} w_{t,h}^I \cdot \text{ReLU} \left(\mathbf{q}_{t,h}^I \cdot K_s^{IComp} \right), \quad (16)$$

$$I_{t,s} = \sum_{h=1}^{nlh} w_{t,h}^I \cdot \text{ReLU}[\mathbf{q}_{t,h}^I \cdot KIComps], (16)$$

$$I_{t,s} =$$

其中， $W^w \in R^{d \times nlh}$ 是可学习矩阵； $w_{t,h}^I \in R$ 是第 h 个 indexer 头的权重。对于查询 Token t ，给定其索引分数 $I_{t,:}$ ，我们使用 top-k 选择器有选择地保留一部分压缩后的 KV 条目 $C_{SprsComp}$ ，用于后续核心注意力：

$$C_t^{SprsComp} = \left\{ C_s^{Comp} \mid I_{t,s} \in \text{Top-k}(I_{t,:}) \right\}. \quad (17)$$



滑动窗口KV 条目

查询 Token 的隐藏状态

KV Token 的隐藏状态

共享键值 MQA。在选出稀疏 KV 条目之后，CSA 随后以多查询注意力（MQA）（Shazeer, 2019）的方式执行核心注意力，其中 $CSprComp$ 中的每个压缩 KV 条目同时充当注意力的键和值。具体来说，对于查询 Token t ，我们首先从压缩潜变量 cQ_t 生成注意力查询 $\{q_{t,1}; q_{t,2}; \dots; q_{t,nh}\}$ ：

$$[q_{t,1}; q_{t,2}; \dots; q_{t,nh}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot \mathbf{W}^{UQ}, \quad (18)$$

其中， nh 表示查询头数量； $\mathbf{W}^{UQ} \in \mathbb{R}^{d_c \times c_{nh}}$ 是查询的上投影矩阵。注意，潜在查询向量 cQ_t 与 indexer query 所使用的潜在向量是共享的。接下来，我们在 $\{q_{t,i}\}$ 和 $CSprComp$ 上执行 MQA：

$$\mathbf{o}_{t,i} = \text{CoreAttn} \left(\text{query} = \mathbf{q}_{t,i}, \text{key} = \mathbf{C}_t^{\text{SprsComp}}, \text{value} = \mathbf{C}_t^{\text{SprsComp}} \right), \quad (19)$$

其中, $ot_{t,i} \in R_c$ 是第 t 个 Token 处第 i 个头的核心注意力输出; $\text{CoreAttn}(\cdot)$ 表示核心注意力操作。

分组输出投影。在 DeepSeek-V4 的配置中, cnh 相当大。因此, 若直接将核心注意力操作的输出 $[ot_{t,1}; ot_{t,2}; \dots; ot_{t,nh}] = ot \in R_{cnh}$ 投影到一个 d 维隐藏状态上, 会带来显著的计算负担。为缓解这一成本, 我们设计了分组输出投影策略。具体来说, 我们首先将 nh 个输出拆分为

g 组, 然后对每组输出 $oG_{t,i} \in R_{cnhg}$, 我们将其投影到一个 d_g 维的中间输出 $oG'_{t,i} \in R_{d_g}$, 其中 $d_g < cnhg$ 。

最后, 我们将中间输出 $[oG'_{t,1}; oG'_{t,2}; \dots; oG'_{t,g}] \in R_{d_g g}$ 投影为最终注意力输出 $\hat{ot} \in R_d$ 。

2.3.2. 重度压缩注意力

HCA 的核心架构如图 4 所示, 它以更重的方式压缩 KV 缓存, 但不使用稀疏注意力。

压缩键值条目。总体而言, HCA 的压缩策略与 CSA 类似, 但采用更大的压缩率 m' ($\gg m$), 且不进行重叠

图 4 | HCA 的核心架构。它执行更重的压缩, 其中 m' ($\gg m$) 个 Token 的 KV 条目会被合并为一个。同时, 我们还额外引入一小组滑动窗口 KV 条目, 以增强局部细粒度依赖。

原文第 12 页

压缩。设 $H \in R_{n \times d}$ 为一段输入隐藏状态序列, HCA 首先计算原始 KV 条目 $C \in R_{n \times c}$ 及其对应的压缩权重 $Z \in R_{n \times c}$:

$$C = H \cdot W^{KV}, \quad (20)$$

$$Z = H \cdot W^Z, \quad (21)$$

$$Z = H \cdot WZ, (21)$$

其中, W^{KV} 、 $W^Z \in R_{d \times c}$ 是可训练参数。接下来, C 中每 m' 个 KV 条目会依据压缩权重以及可学习的位置偏置 $B \in R_{m' \times c}$ 被压缩为一个条目, 生成 $C_{\text{Comp}} \in R_{nm' \times c}$ 。每个压缩条目 $C_{\text{Comp}i} \in R_c$ 的计算方式为

$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m'i:m'(i+1)-1} + B), \quad (22)$$

$$m'(i+1)-1 \sum$$

$CComp_i =$

$$\sum_{j=m'i}^{m'i+m'-1} S_j \odot C_j. \quad (23)$$

通过这一压缩操作，HCA 将序列长度压缩到原来的 $1/m'$

倍。

共享键值 MQA 与分组输出投影。HCA 同样采用与 CSA 相同的共享 KV MQA 和分组输出投影策略。完成 KV 压缩后，对于查询 Token t ，HCA 首先以低秩方式生成注意力查询 $\{q_{t,1}; q_{t,2}; \dots; q_{t,nh}\}$:

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ}, \quad (24)$$

$$[\mathbf{q}_{t,1}; \mathbf{q}_{t,2}; \dots; \mathbf{q}_{t,nh}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (25)$$

$$[q_{t,1}; q_{t,2}; \dots; q_{t,nh}] = \mathbf{q}_t = \mathbf{c}_t^Q \cdot W^{UQ}, \quad (25)$$

其中， $\mathbf{h}_t \in R_d$ 是查询 Token t 的输入隐藏状态； nh 表示查询头数量； $W^{DQ} \in R_{d \times d_c}$ 和 $W^{UQ} \in R_{d_c \times cnh}$ 分别是查询的下投影和上投影矩阵。接下来，我们在 $\{q_{t,i}\}$ 和 $CComp$ 上执行 MQA:

$$\mathbf{o}_{t,i} = \text{CoreAttn}(\text{query}=\mathbf{q}_{t,i}, \text{key}=\mathbf{C}^{\text{Comp}}, \text{value}=\mathbf{C}^{\text{Comp}}), \quad (26)$$

其中， $\mathbf{o}_{t,i} \in R_c$ 是第 t 个 Token 处第 i 个头的核心注意力输出。接下来，与 CSA 一样，

HCA 将 nh 个输出划分为 g 组，并对每组输出 $\mathbf{o}_{Gt,i} \in R_{cnh}$

g ，HCA 将其投影到一个 dg 维的中间输出 $\mathbf{o}_{G't,i} \in R_{dg}$ ，其中 $dg < cnh$

g 。最后，HCA 将中间输出 $[\mathbf{o}_{G't,1}; \mathbf{o}_{G't,2}; \dots; \mathbf{o}_{G't,g}] \in R_{dgg}$ 投影为最终注意力输出 $\hat{\mathbf{o}}_t \in R_d$ 。

2.3.3. 其他细节

除上述 CSA 与 HCA 的核心架构外，我们的混合注意力还融合了若干其他技术。为使表述更清晰，我们在上面的介绍中省略了这些附加技术，并将在本小节中简要说明。此外，本小节仅关注

这些技术的核心思想，为了简洁起见，可能会省略一些细微细节。我们鼓励读者参考我们的开源实现，以获取无歧义的细节说明。

查询与键值条目归一化。对于 CSA 和 HCA，我们都会在核心注意力操作之前，对查询的每个头以及压缩 KV 条目的唯一头额外执行一次 RMSNorm 操作。这种归一化可以避免注意力 logits 爆炸，并可能提升训练稳定性。

原文第 13 页

部分旋转位置嵌入。对于 CSA 和 HCA，我们在注意力查询、KV 条目以及核心注意力输出上部分采用旋转位置嵌入 (RoPE) (Su et al., 2024)。具体来说，对于 CSA 和 HCA 中使用的每个查询向量和 KV 条目向量，我们会在其最后 64 个维度上应用 RoPE。由于 KV 条目同时充当注意力键和值，朴素的核心注意力输出 $\{ot_i\}$ 会携带由 KV 条目加权和带来的绝对位置嵌入。作为对策，我们还会在每个 ot_i 的最后 64 个维度上，以位置 $-i$ 应用 RoPE。这样一来，核心注意力的输出也会携带相对位置嵌入——每个 KV 条目对核心注意力输出的贡献也将与查询和该 KV 条目之间的距离相关。

滑动窗口注意力的附加分支。为了在 CSA 和 HCA 中严格保持因果性，每个查询只能关注其之前的压缩 KV 块。因此，查询无法访问其自身压缩块内其他 Token 的信息。与此同时，在语言建模中，最近的 Token 往往与查询 Token 具有更高相关性。基于这些原因，我们为 CSA 和 HCA 都引入了一个以滑动窗口方式运行的补充注意力分支，以更好地建模局部依赖。具体来说，对于每个查询 Token，我们还会额外生成与最近 $nwin$ 个 Token 对应的 $nwin$ 个未压缩 KV 条目。在 CSA 和 HCA 的核心注意力中，这些滑动窗口中的 KV 条目会与压缩 KV 条目一起使用。

Attention Sink。在 CSA 与 HCA 的核心注意力中，我们采用了 attention sink 技巧 (OpenAI, 2025; Xiao et al., 2024)。具体来说，我们设置一组可学习的 sink logits $\{z'_1, z'_2, \dots, z'_{nh}\}$ 。对于第 h 个注意力头， $\text{Exp}(z'_h)$ 会被加入到注意力分数的分母中：

$$s_{h,i,j} = \frac{\text{Exp}(z_{h,i,j})}{\sum_k \text{Exp}(z_{h,i,k}) + \text{Exp}(z'_h)}, \quad (27)$$

其中， $s_{h,i,j}$ 、 $z_{h,i,j} \in \mathbb{R}$ 分别表示第 h 个注意力头中，第 i 个查询 Token 与第 j 个先前 Token 或压缩块之间的注意力分数和注意力 logit。该技术使得每个查询头可以将其总注意力分数调整为不必等于 1，甚至可以接近 0。

2.3.4. 效率讨论

由于采用了混合 CSA 与 HCA，再加上低精度计算与存储，DeepSeek-V4 系列的注意力模块在注意力 FLOPs 和 KV 缓存大小方面都实现了显著效率提升，尤其是在长上下文场景中。首先，我们对 KV 条目采用混合存储格式：旋转位置嵌入（RoPE）对应维度使用 BF16 精度，其余维度使用 FP8 精度。这种混合表示相比纯 BF16 存储可将 KV 缓存大小减少近一半。其次，lightning indexer 内部的注意力计算采用 FP4 精度，这能在超长上下文下加速注意力操作。第三，相比 DeepSeek-V3.2，DeepSeek-V4 系列选择了更小的 attention top-k，从而提升模型在短、中等长度文本上的效率。最后，也是最重要的一点，压缩注意力与混合注意力技术大幅减少了 KV 缓存大小和计算 FLOPs。

以头维度为 128 的 BF16 GQA8 (Ainslie et al., 2023) 作为基线——这是大语言模型注意力中的一种常见配置——在 1M 上下文设定下，DeepSeek-V4 系列的 KV 缓存大小可大幅降低到该基线的大约 2%。

原文第 14 页

算法 1 DeepSeek-V4 的 Muon 优化器

输入：学习率 η 、动量 μ 、权重衰减 λ 、更新重缩放因子 γ

1: 对每个训练步骤 t 执行

2: 对每个逻辑上独立的权重 $W \in \mathbb{R}^{n \times m}$ 执行

3: $G_t = \nabla W L_t(W_{t-1})$ $\triangleleft$ 计算梯度

4: $M_t = \mu M_{t-1} + G_t$ $\triangleleft$ 累积动量缓冲区

5: $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$ $\triangleleft$ Nesterov 技巧与混合 Newton-Schulz

6: $O_t = O'_t \cdot \sqrt{\cdot}$

$\max(n, m) \cdot \gamma$ $\triangleleft$ 重缩放更新的 RMS

7: $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$ $\triangleleft$ 执行权重衰减与参数更新

8: end for

9: end for

此外，即便与已经相当高效的基线 DeepSeek-V3.2 (DeepSeek-AI, 2025) 相比，DeepSeek-V4 系列在效率方面仍表现出显著优势。它们的推理 FLOPs 与 KV 缓存大小对比见图 1 右侧。

2.4. Muon 优化器

由于收敛更快、训练稳定性更好，我们在 DeepSeek-V4 系列的大多数模块中采用了 Muon (Jordan et al., 2024; Liu et al., 2025) 优化器。我们使用的 Muon 优化完整算法汇总于算法 1。

基础配置。对于嵌入模块、预测头模块、mHC 模块中的静态偏置与门控因子，以及所有 RMSNorm 模块的权重，我们仍然保留 AdamW (Loshchilov and Hutter, 2017) 优化器。其他所有模块都使用 Muon 更新。遵循 Liu et al. (2025)，我们同样对 Muon 参数施加权重衰减、采用 Nesterov (Jordan et al., 2024; Nesterov, 1983) 技巧，并对更新矩阵的均方根 (RMS) 进行重缩放，以复用我们的 AdamW 超参数。与他们不同的是，我们使用混合 Newton-Schulz 迭代来进行正交化。

混合 Newton-Schulz 迭代。对于给定矩阵 M ，设其奇异值分解 (SVD) 为 $M = U\Sigma VT$ 。Newton-Schulz 迭代的目的是将 M 近似正交化为 UVT 。通常，首先会将 M 归一化为 $M_0 = M/\|M\|_F$ ，以确保其最大奇异值不超过 1。随后，每一次 Newton-Schulz 迭代执行如下操作：

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2M_{k-1}. \quad (28)$$

我们的混合 Newton-Schulz 分两个不同阶段共执行 10 次迭代。在前 8 步中，我们使用系数 $(a, b, c) = (3.4445, -4.7750, 2.0315)$ 以实现快速收敛，使奇异值接近 1。在最后 2 步中，我们切换为系数 $(a, b, c) = (2, -1.5, 0.5)$ ，从而将奇异值稳定在 1。

避免注意力 logits 爆炸。DeepSeek-V4 系列的注意力架构使我们能够直接对注意力查询和 KV 条目应用 RMSNorm，这可以有效防止注意力 logits 爆炸。因此，我们在 Muon 优化器中不使用 QK-Clip 技术 (Liu et al., 2025)。

原文第 15 页

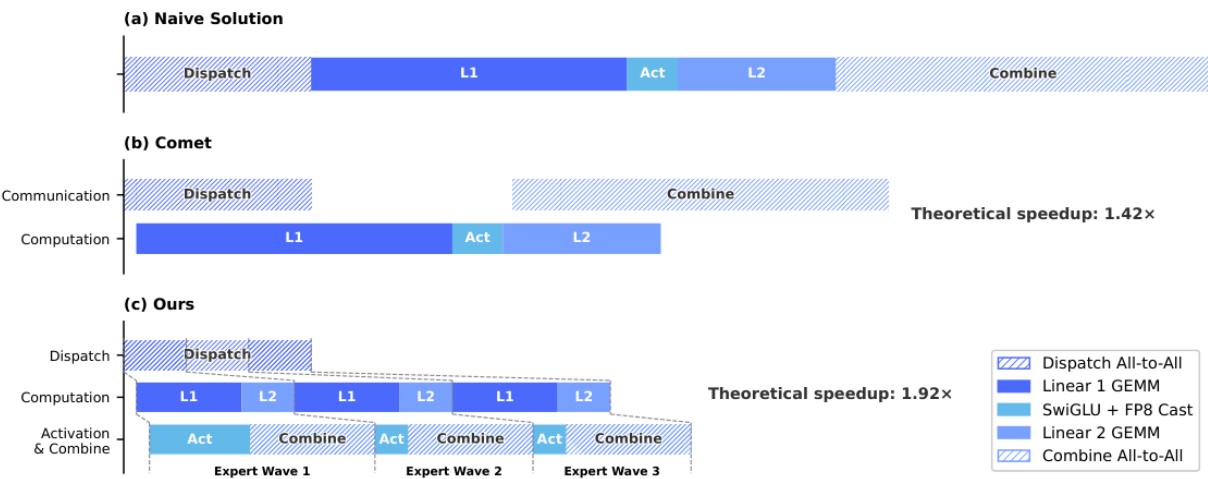
3. 通用基础设施

3.1. 专家并行中的细粒度通信—计算重叠

专家混合 (MoE) 可以通过专家并行 (EP) 加速。然而，EP 需要复杂的跨节点通信，并对互连带宽和时延提出很高要求。为缓解 EP 中的通信瓶颈，并在更低互连带宽要求下实现更高的端到端性能，我们提出了一种细粒度 EP 方案，将通信与计算融合进单一流水化内核中，以实现通信—计算重叠。

通信时延可以被隐藏。我们这一 EP 方案的核心洞见是：在 MoE 层中，通信时延可以有效地被计算掩蔽。如图 5 所示，在 DeepSeek-V4 系列中，每个 MoE 层主要可以分解为四个阶段：两

个通信受限阶段 Dispatch 和 Combine，以及两个计算受限阶段 Linear-1 和 Linear-2。我们的性能分析显示，在单个 MoE 层内，总通信时间小于总计算时间。因此，在将通信与计算融合为统一流水线之后，计算仍然是主要瓶颈，这意味着系统能够在不降低端到端性能的情况下容忍更低的互连带宽。



细粒度 EP 方案。为进一步降低互连带宽需求并放大重叠带来的收益，我们引入了一种更细粒度的专家划分方案。受多项相关工作启发 (Aimuyo et al., 2025; Zhang et al., 2025b)，我们将专家拆分并按波次 (waves) 调度。每个波次由一小部分专家组成。一旦该波次中的所有专家完成通信，就可以立即开始计算，而无需等待其他专家。在稳态下，当前波次的计算、下一波次的 Token 传输以及已完成专家的结果发送会同时进行，如图 5 所示。这就在专家之间形成了一个细粒度流水线，使得整个波次期间计算与通信都能持续进行。基于波次的调度会加速

图 5 | 我们的 EP 方案与相关工作示意图。Comet (Zhang et al., 2025b) 分别将 Dispatch 与 Linear-1 重叠、将 Linear-2 与 Combine 重叠。我们的 EP 方案通过将专家拆分并按波次调度，实现了更细粒度的重叠。理论加速比是在 DeepSeek-V4-Flash 架构配置下评估的。

诸如强化学习 (RL) rollout 等极端场景下的性能提升，这类场景通常会遇到长尾小批次。

性能与开源 Mega-Kernel。我们在 NVIDIA GPU 和 HUAWEI Ascend NPU 平台上验证了细粒度 EP 方案。与强大的非融合基线相比，它在通用推理工作负载上实现了 1.50~1.73x 的加速，在 RL rollout 和高速智能体服务等对时延敏感的场景中最高可达 1.96x。我们已将基于 CUDA 的 mega-kernel 实现以 MegaMoE2 的名称作为 DeepGEMM 的一个组件开源。

观察与建议。我们分享了在内核开发中的观察与经验，并向硬件厂商提出一些建议，希望有助于高效硬件设计并实现更好的软硬件协同设计：

- 计算—通信比。完全的通信—计算重叠取决于计算—通信比，而不是单纯取决于带宽。记峰值计算吞吐量为 C 、互连带宽为 B ，当 $C/B \leq V_{\text{comp}}/V_{\text{comm}}$ 时，通信可以被完全隐藏，其中 V_{comp} 表示计算量， V_{comm} 表示通信量。对于 DeepSeek-V4-Pro，每个 token-expert 对需要 $6hd$ FLOPs（SwiGLU gate、up 和 down 投影），但只需要 $3h$ 字节通信（FP8 Dispatch + BF16 Combine），于是可简化为：

$$C$$
$$B \leq 2d = 6144 \text{ FLOPs/Byte.}$$

也就是说，每 1 GBps 的互连带宽就足以隐藏 6.1 TFLOP/s 计算所对应的通信。一旦带宽达到这一阈值，它就不再是瓶颈，再投入更多硅面积继续提升带宽的边际收益会迅速下降。我们鼓励未来的硬件设计瞄准这种平衡点，而不是无条件地扩展带宽。

- 功耗预算。极端的内核融合会使计算、内存和网络同时处于高负载状态，从而使功耗限频成为关键性能限制因素。我们建议未来的硬件设计为这种完全并发的 workload 预留足够的功耗余量。
- 通信原语。我们采用基于拉取（pull）的方式，由每块 GPU 主动从远端 GPU 读取数据，从而避免细粒度 push 所带来的高通知时延。若未来硬件能够提供更低时延的跨 GPU 信令，那么 push 将变得可行，并支持更自然的通信模式。
- 激活函数。我们建议用一种低成本的逐元素激活函数替代 SwiGLU，该函数不涉及指数或除法运算。这样可以直接减轻 GEMM 之后的处理开销；同时，在相同参数预算下，移除 gate 投影还能增大中间维度 d ，从而进一步放宽带宽要求。

3.2. 使用 TileLang 的灵活高效内核开发

在实践中，我们精细的模型架构本来会产生数百个细粒度的 Torch ATen 算子。我们采用 TileLang (Wang et al., 2026) 开发了一组融合内核来替代其中绝大多数，以最小投入实现最优性能。它

2<https://github.com/deepseek-ai/DeepGEMM/pull/304>

原文第 17 页

也使我们能够在验证期间快速原型化诸如注意力变体等算子。这些内核在模型架构开发、大规模训练以及最终推理服务的生产部署中都发挥着关键作用。作为一种领域专用语言（DSL），TileLang 在开发效率与运行时效率之间取得平衡，使得快速开发与在同一代码库中进行深入、迭代式优化成为可能。此外，我们还与 TileLang 社区紧密合作，推动形成更敏捷、更高效、更稳定的内核开发工作流。

通过 Host Codegen 降低调用开销。随着加速器性能持续提升，CPU 侧编排开销变得越来越突出。对于小型且高度优化的内核而言，这类固定的主机端开销很容易限制利用率和吞吐量。这种

开销的一个常见来源是：主机端逻辑（例如运行时契约检查）通常为了灵活性而以 Python 编写，因此会带来固定的每次调用成本。

我们通过 Host Codegen 来缓解这类开销，其做法是将大多数主机端逻辑迁移到生成的主机代码中。具体而言，我们首先在 IR（中间表示）层面联合生成设备端内核和轻量级主机启动器，并嵌入从语言前端解析得到的必要元数据——如数据类型、rank/shape 约束以及 stride/layout 假设。随后，这个启动器会被降级为基于 TVM-FFI (Chen et al., 2018) 框架的主机源代码，其紧凑的调用约定与零拷贝张量互操作共同将主机端开销降到最低。在运行时，这些生成的主机代码负责执行校验与参数编组，从而将所有逐次调用检查移出 Python 执行路径。我们的测量表明，CPU 侧校验开销从每次调用几十到几百微秒降到了不足一微秒。

借助 SMT 求解器的形式化整数分析。TileLang 内核涉及复杂的张量索引算术，这需要强大的形式化整数分析能力。在编译过程中的布局推断、内存风险检测和边界分析等阶段，编译器必须验证整数表达式是否满足特定性质，才能启用相应优化。因此，更强的形式化分析能力可以释放更高级、更复杂的优化机会。

为此，我们将 Z3 SMT 求解器 (De Moura and Bjørner, 2008) 集成到 TileLang 的代数系统中，为张量程序中的大多数整数表达式提供形式化分析能力。我们通过将 TileLang 的整数表达式翻译为 Z3 的无量词非线性整数算术 (QF_NIA)，在计算开销与形式表达能力之间取得平衡。基于整数线性规划 (ILP) 求解器，QF_NIA 可以无缝处理内核中常见的标准线性整数表达式。此外，其固有的非线性推理能力还可有效应对诸如可变张量形状上的向量化等高级挑战。在合理的资源限制下，Z3 能够提升整体优化性能，同时将编译时间开销限制在几秒钟之内。其影响覆盖多个编译阶段，包括向量化、barrier 插入和代码简化。

数值精度与按位可复现性。在生产环境中，数值正确性与可复现性与原始吞吐量同样关键。因此，我们默认优先保证精度：在编译器层面禁用 fast-math 优化，而影响精度的近似仅以显式、可选加入的前端算子形式提供（例如 `T._exp`、`T._log` 和 `T._sin`）。相反，当需要严格的 IEEE-754 语义时，TileLang 提供

具有显式舍入模式的 IEEE 合规 intrinsic（例如 `T.ieee_fsqrt`、`T.ieee_fdiv` 和 `T.ieee_add`），使开发者能够精确指定数值行为。

我们还将按位可复现性作为目标，以便将内核与手写 CUDA 基线进行验证。我们使 TileLang 的代数化简与 lowering 规则与主流 CUDA 工具链（例如 NVCC）保持一致，避免引入非预期位级差异的变换。布局注解（例如 `T.annotate_layout`）还允许用户固定依赖布局的 lowering 决策，使求值与累加顺序与参考 CUDA 实现保持一致，从而在需要时实现按位完全一致的输出。

我们的评测表明，这些以精度和可复现性为导向的设计选择并不会牺牲性能：在保守默认设置下，TileLang 内核仍然具有竞争力，同时也暴露出可选择性放宽数值约束以获得更高速度的调节项。

3.3. 高性能、批不变且确定性的内核库

为了支持高效训练与推理，我们开发了一整套高性能计算内核。除了基础功能和最大化硬件利用率之外，另一个关键设计目标是确保预训练、后训练与推理流水线之间的训练可复现性和按位对齐。因此，我们以极低的性能开销实现了端到端、按位级别的批不变且确定性的内核。这些内核对调试、稳定性分析以及后训练阶段的一致行为都很有帮助。

批不变性。批不变性确保任意给定 Token 的输出，无论它在一个 batch 中所处位置如何，都保持按位完全一致。为了实现批不变性，主要挑战如下：

- 注意力。为了实现批不变性，我们不能使用 split-KV 方法 (Dao et al., 2023)，该方法会将单个序列的注意力计算分布到多个流式多处理器 (SM) 上，以平衡 SM 负载。然而，放弃这一技术会导致严重的 wave quantization 问题³，从而不利于 GPU 利用率。为了解决这一问题，我们为批不变解码开发了双内核策略。第一个内核在单个 SM 内完成整段序列的注意力输出计算，在 wave 被完全占满时可确保高吞吐量。第二个内核为了最小化最后一个未完全填满 wave 的时延，从而缓解 wave quantization，会对单个序列使用多个 SM。为了保证这两个内核在按位层面一致，我们精心设计了第二个内核的计算路径，使其累加顺序与第一个内核相同。此外，第二个内核还在 thread-block cluster 内使用了分布式共享内存⁴，从而支持跨 SM 的高速数据交换。这种双内核方法将批不变解码的额外开销有效限制在可忽略范围内。
- 矩阵乘法。传统 cuBLAS 库 (NVIDIA Corporation, 2024) 无法实现批不变性。因此，我们用 DeepGEMM (Zhao et al., 2025) 在端到端上替代了它。此外，对于非常小的 batch size，传统实现通常会采用 split-k (Osama et al., 2023) 技术来提升性能。不幸的是，split-k 技术无法保证批不变性，而这正是 DeepSeek-V4 中的关键特性。

³<https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#wave-quant>
⁴<https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#distributed-shared-memory>

因此，我们在大多数场景中放弃了 split-k，但这可能会导致性能下降。为了解决这一问题，我们引入了一系列优化，使我们的矩阵乘法实现能够在大多数主要场景中达到甚至超过标准 split-k 的性能。

确定性。确定性训练对于调试硬件或软件问题非常有益。此外，当训练过程中出现如 loss spike 等异常时，确定性可以帮助研究人员更容易定位数值原因，并进一步改进模型设计。训练中的非

确定性通常源于非确定性的累加顺序，而这往往又是由原子加法指令的使用导致的。这个问题主要出现在反向传播阶段，尤其集中在以下部分：

- 注意力反向传播。在传统的稀疏注意力反向传播实现中，我们使用 `atomicAdd` 来累加 KV Token 的梯度。这会由于浮点加法的非结合性而引入非确定性。为解决这一问题，我们为每个 SM 分配独立的累加缓冲区，随后对所有缓冲区执行全局确定性求和。
- MoE 反向传播。当来自不同 rank 的多个 SM 同时向接收端 rank 上的同一缓冲区写入数据时，写入位置的协商也会引入非确定性。为了解决这一问题，我们在每个单独 rank 内设计了 Token 顺序预处理机制，并结合跨多个 rank 的缓冲区隔离策略。这一策略确保了专家并行发送结果的确定性，以及 MoE 反向传播中累加顺序的确定性。
- mHC 中的矩阵乘法。mHC 涉及一个输出维度仅为 24 的矩阵乘法。对于非常小的 batch size，我们不得不使用 `split-k` (Osama et al., 2023) 算法，而其朴素实现会导致非确定性。为克服这一点，我们将每个 `split` 部分分别输出，并在后续内核中进行确定性归约，从而同时保留性能与确定性。

3.4. FP4 感知量化训练

为了在部署时实现推理加速和内存节省，我们在后训练阶段引入量化感知训练 (QAT) (Jacob et al., 2018)，使模型能够适应量化带来的精度退化。我们将 FP4 (MXFP4) 量化 (Rouhani et al., 2023) 应用到两个部分：(1) MoE 专家权重，它是 GPU 内存占用的主要来源之一 (OpenAI, 2025)；(2) CSA 的 indexer 中的 Query-Key (QK) 路径，其中 QK 激活会完全以 FP4 形式缓存、加载并进行乘法计算，从而在长上下文场景中加速注意力分数计算。此外，在这一 QAT 过程中，我们还将索引分数 $I_{:,}$ 从 FP32 进一步量化为 BF16。该优化使 top-k 选择器获得了 2× 加速，同时保持了 99.7% 的 KV 条目召回率。

对于 MoE 专家权重，遵循 QAT 的常见做法，优化器维护的 FP32 主权重会先量化为 FP4，然后再反量化回 FP8 以参与计算。值得注意的是，我们的 FP4 到 FP8 反量化是无损的。这是因为 FP8 (E4M3) 相比 FP4 (E2M1) 多出 2 个指数位，具有更大的动态范围。因此，只要每个 FP8 量化块 (128×128 tiles) 内的 FP4 子块 (1×32 tiles) 之间最大与最小 scale factor 的比值不超过某个阈值，细粒度的 scale 信息就可以被 FP8 扩展后的动态范围完全吸收。我们通过实验验证，当前权重满足这一条件。这使得整个 QAT 流水线能够完全复用现有的 FP8 训练框架，而无需

任何修改。在反向传播阶段，梯度是相对于前向传播中使用的同一份 FP8 权重来计算的，并直接回传到 FP32 主权重，这等价于通过量化操作应用 Straight-Through Estimator (STE)。这也避免了对转置权重重新量化的需要。

在 RL 训练的推理与 rollout 阶段，由于不涉及反向传播，我们直接使用真实的 FP4 量化权重，而不是模拟量化。这样可以确保采样时模型行为与在线部署完全一致，同时也减少了内核的内存加载，从而获得实际加速，并显著降低内存消耗。我们以类似方式处理 CSA 的 indexer 中的 QK 路径。

3.5. 训练框架

我们的训练框架建立在为 DeepSeek-V3 (DeepSeek-AI, 2024) 开发的可扩展且高效的基础设施之上。在训练 DeepSeek-V4 时，我们继承了这一稳健基础，同时引入了若干关键创新，以适配其新的架构组件——特别是 Muon 优化器、mHC 和混合注意力机制——并保持高训练效率与稳定性。

3.5.1. Muon 的高效实现

Muon 优化器需要完整的梯度矩阵来计算参数更新，这在与 Zero Redundancy Optimizer (ZeRO) (Rajbhandari et al., 2020) 结合时会带来挑战。传统 ZeRO 是为 AdamW 这类逐元素优化器设计的，在那种情况下，单个参数矩阵可以在多个 rank 之间拆分并更新。为解决这一冲突，我们为 Muon 设计了一种 ZeRO bucket 分配的混合策略。

对于稠密参数，我们限制 ZeRO 并行的最大规模，并使用背包算法将参数矩阵分配到这些 rank 上，确保每个 rank 承担的负载大致均衡。每个 rank 上的 bucket 都会被填充到与各 rank 中最大 bucket 相同的大小，以便高效执行 reduce-scatter 操作。在我们的设置中，每个 rank 管理的参数矩阵不超过五个，这种填充通常只带来不到 10% 的内存开销。当数据并行的整体规模超过 ZeRO 的上限时，我们会在额外的数据并行组上冗余计算 Muon 更新，用计算换取更低的总 bucket 内存。

对于 MoE 参数，我们对每个专家分别进行优化。我们首先将所有层中所有专家的 SwiGLU (Shazeer, 2020) 里的 down projection 矩阵展平，随后再展平 up projection 矩阵和 gate 矩阵。然后，我们对展平后的向量进行填充，以确保能够在不拆分任何逻辑独立矩阵的前提下，将该向量均匀分布到所有 rank 上。由于专家数量很多，我们对 MoE 参数不设 ZeRO 并行规模上限，填充开销也可以忽略不计。

此外，在每个 rank 上，形状相同且相邻的参数会被自动合并，从而支持对 Newton-Schulz 迭代进行批量执行，以获得更好的硬件利用率。进一步地，我们观察到 Muon 中的 Newton-Schulz 迭代在使用 BF16 矩阵乘法计算时仍然保持稳定。基于这一点，我们进一步以随机舍入的方式，将需要在数据并行 rank 间同步的 MoE 梯度量化到 BF16 精度，从而将通信量减半。为了避免低精度加法器带来的累积误差，我们用一种两阶段方法替代了传统的基于树或环的 reduce-scatter collective。首先，通过 all-to-all 操作在各 rank 之间交换本地梯度；然后，每个 rank 在本地以 FP32 进行求和。该设计保持了数值鲁棒性。

3.5.2. mHC 的低成本且内存高效实现

与传统残差连接相比，引入 mHC 会增加激活内存消耗以及流水线阶段之间的通信量。为缓解这些成本，我们实现了若干优化策略。

首先，我们为训练和推理都精心设计并实现了 mHC 的融合内核。其次，我们引入了一种重计算策略，对中间张量进行选择性的检查点保存。具体来说，我们会重计算层间的大多数隐藏状态以及所有归一化后的层输入，同时避免对计算密集型操作进行重计算。这在节省内存和计算开销之间实现了平衡。第三，我们调整了 DualPipe 1F1B 重叠方案，以适应增加的流水线通信，并支持 mHC 中部分操作的并发执行。

总体而言，这些优化将 mHC 的 wall-time 开销限制在重叠 1F1B 流水线阶段时间的 6.7%。更多工程优化细节可参见专门的 mHC 论文 (Xie et al., 2026)。

3.5.3. 用于长上下文注意力的上下文并行

传统上下文并行 (CP) 会对序列维度进行切分，每个 rank 维护连续的 s 个 Token。这给我们的压缩注意力机制 (即 CSA 和 HCA) 带来了两个挑战。一方面，训练样本是由多个序列打包而成的，而每个序列都会独立地按 m (或 m') 倍进行压缩，任何末尾少于 m 个的 Token 都会被丢弃。因此，压缩后的 KV 长度通常小于 sm ，并且在不同 rank 间会有所变化。另一方面，压缩需要连续的 m 个 KV 条目，而这些条目可能跨越两个相邻 CP rank 的边界。

为了解决这些挑战，我们设计了一种两阶段通信方法。在第一阶段，每个 rank i 会将其最后 m 个未压缩 KV 条目发送给 rank $i + 1$ 。随后，rank $i + 1$ 会将接收到的其中一部分条目与其本地的 s 个未压缩 KV 条目一起压缩，从而生成固定长度的 s

$m + 1$ 个压缩条目，其中包含一些填充条目。在第二阶段，通过在所有 CP rank 之间执行 all-gather 操作，收集各自本地压缩后的 KV 条目。随后，一个融合的 select-and-pad 算子会将它们重新组织为完整的压缩 KV 条目集合，总长度为 $cp_size \cdot s$

m 。所有填充条目都会被放在尾部。对于 HCA 和 CSA 中的 indexer，每个查询 Token 可见的压缩 KV 条目范围可以通过规则预先计算。对于 CSA 中的稀疏注意力，top- k 选择器会显式指定每个查询可见的压缩 KV 条目索引。

3.5.4. 用于灵活激活检查点的扩展自动微分

传统的激活检查点实现以整个模块为粒度，决定在反向传播期间保留还是重计算其输出激活。这种粗粒度通常会导致重计算成本与激活内存占用之间的折中不够理想。另一种方法是手动实现整

个层的前向与反向逻辑，并显式管理张量检查点状态。尽管这种方法可以提供细粒度控制，但会失去自动微分框架带来的便利，从而显著增加开发复杂度。

为了在不牺牲编程效率的前提下实现细粒度控制，我们实现了一个支持自动微分的张量级激活检查点机制。借助该机制，开发者只需实现前向过程，并有选择地标注

原文第 22 页

各个张量，以便自动进行检查点保存和重计算。我们的框架利用 TorchFX (Reed et al., 2022) 追踪完整计算图。对于每个被标注的张量，它会执行反向遍历，以识别重计算该张量所需的最小子图。我们将这些最小子图定义为重计算图，并在相应梯度计算之前，将它们插入到反向逻辑中。

与手工实现相比，这一设计在训练期间不会引入额外开销。该框架中的重计算是通过直接释放被标注张量的 GPU 内存，并复用重计算张量的存储指针来实现的，不涉及任何 GPU 内存拷贝。此外，由于图追踪会对模型进行具体执行，我们可以追踪每个张量底层的存储指针，从而能够自动对共享存储的张量（例如 reshape 操作的输入与输出）进行重计算去重。这使开发者在标注重计算时无需推理底层内存细节。

3.6. 推理框架

我们的推理框架大体上继承自 DeepSeek-V3，但在 KV Cache 管理方面有所不同。

3.6.1. KV Cache 结构与管理

为了高效管理 DeepSeek-V4 中混合注意力机制带来的异构 KV 缓存，我们设计了定制化的 KV cache 布局。该布局如图 6 所示，下面将对其进行详细说明。

DeepSeek-V4 中的异构 KV 条目。DeepSeek-V4 系列中的混合注意力机制引入了多种不同类型的 KV 条目，它们具有不同的键值 (KV) 缓存大小和更新规则。用于稀疏选择的 lightning indexer 在 KV cache 中引入了额外维度，其嵌入大小与主注意力中的嵌入大小不同。CSA 和 HCA 所采用的压缩技术将序列长度分别压缩为原来的 $\frac{1}{m}$ 和 $\frac{1}{m'}$

m 和 1

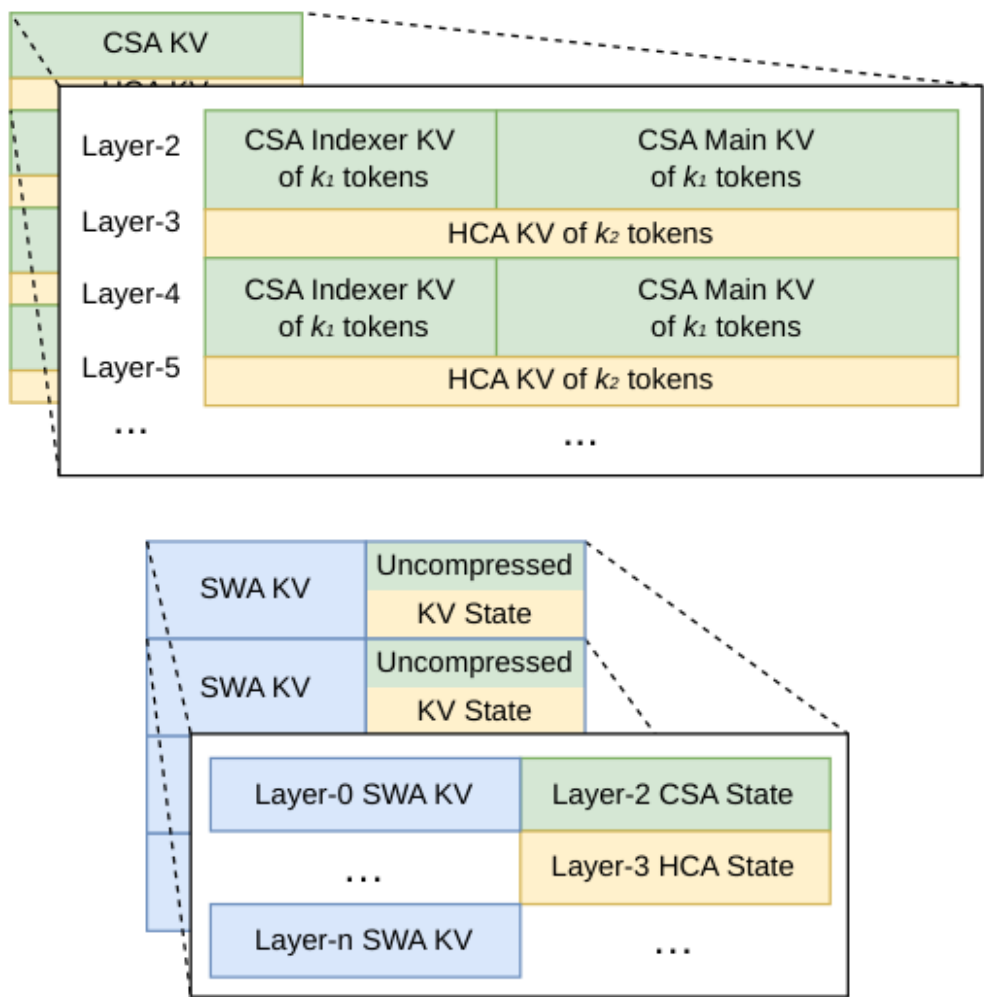
m' ，从而降低整体 KV cache 大小。因此，不同层之间的 KV cache 大小会有所差异。此外，滑动窗口注意力 (SWA) 层也使用不同的 KV cache 大小，以及独立的缓存命中与淘汰策略。在压缩分支中，每 m 个 Token 会生成一个 KV 条目。当剩余 Token 数量不足以执行压缩时，所有待处理的 Token 及其相关隐藏状态都必须保存在缓冲区中，直到可以执行压缩操作。这些缓冲的 Token 代表一种由位置上下文决定的序列状态，也会在 KV cache 框架内进行管理。

管理混合注意力 KV Cache 的挑战。混合注意力机制违背了 PagedAttention 及其变体背后的基本假设。尽管近期的一些混合 KV cache 管理算法（例如 Jenga（Zhang et al., 2025a）、Hymba（Dong et al., 2025））面向通用混合注意力模型或特定结构，但在 PagedAttention 框架下将所有层的 KV cache 统一起来仍有两个主要障碍：

- 多样化的缓存策略，例如滑动窗口注意力中使用的缓存策略。
- 高性能注意力内核施加的约束，包括对齐要求。

状态缓存

KV Cache



请求 1

块 0

请求 2

块 1

请求 3

块 2

.....

.....

未压缩

CSA KV

请求 R

SWA KV

块 N

KV 状态

HCA KV

m CSA个压缩 Token, 以及 $k_2 = \text{lcm}(m, m')$

m' HCA 个压缩 Token。

为了高效管理 DeepSeek-V4 的 KV cache，我们设计了对应策略来克服这两个挑战。

SWA 与未压缩尾部 Token 的状态缓存。为了解决第一个障碍，我们采用了一种替代性的缓存管理机制。由于 SWA 的设计目标是在受限 KV cache 大小下提升性能，因此将其与压缩分支中的未压缩尾部 Token 一并视作状态空间模型是合理的。相应的 KV cache 因而可以被看作一种仅依赖当前位置的序列特定状态。据此，我们预先分配了一个固定且有限大小的状态缓存池，并动态地将其分配给每条序列。

稀疏注意力内核协同设计。针对第二个障碍，传统高性能注意力内核通常假设每个块有固定数量 B 的 Token，以优化性能；这在 CSA 中对应于 $B \cdot m$ 个原始 Token，在 HCA 中对应于 $B \cdot m'$ 个原始 Token。通过采用高性能稀疏注意力内核，不同层可以在不损失性能的情况下容纳每块可变数量的 Token。要实现这一点，需要对 KV cache 布局 and 稀疏注意力内核进行协同设计。例如，将块填充到与 cache line 对齐可以提升性能。因此，对于压缩率为 m 的 CSA 和压缩率为 m' 的

HCA，每个块所覆盖的原始 Token 数量可以是 $\text{lcm}(m, m')$ ——即这两个压缩率最小公倍数——的任意倍数。

3.6.2. 磁盘上的 KV Cache 存储

在提供 DeepSeek-V4 服务时，我们利用磁盘上的 KV cache 存储机制来消除共享前缀请求的重复 prefilling。对于 CSA/HCA 中压缩后的 KV 条目，以及滑动窗口注意力（SWA）中的未压缩 KV 条目，我们分别设计了不同的存储管理方案。

图 6 | DeepSeek-V4 的 KV cache 布局示意图。KV cache 被组织为两个主要组成部分：用于 CSA/HCA 的经典 KV cache，以及用于 SWA 和 CSA/HCA 中尚未可压缩 Token 的状态 cache。在状态 cache 中，每个请求都会分配一个固定大小的缓存块。在该块内，SWA 段存储最近 n_{win} 个 Token 对应的 KV 条目，而 CSA/HCA 段存储尚未准备好压缩的未压缩尾部状态。在经典 KV cache 中，我们为每个请求分配多个块。每个缓存块覆盖 $\text{lcm}(m, m')$ 个原始 Token，并生成 $k1 = \text{lcm}(m, m')$

原文第 24 页

对于 CSA 和 HCA，我们直接将所有压缩后的 KV 条目存储到磁盘上。当某个请求命中已存储前缀时，我们会读取并复用与该前缀对应的压缩 KV 条目，一直读到最后一个完整压缩块为止。特别地，对于尾部不完整块中的前缀 Token，我们仍需要重新计算它们，以恢复未压缩 KV 条目，因为 CSA 与 HCA 中的未压缩 KV 条目并未被存储。

对于 SWA 的 KV 条目，由于它们未经压缩且存在于每一层中，其体量大约是压缩后的 CSA 和 HCA KV 条目的 8 倍。为了高效处理这些体量巨大的 SWA KV 条目，我们提出并实现了三种不同的磁盘 SWA KV 管理策略，它们在存储开销和计算冗余之间提供了不同权衡：

- 完整 SWA 缓存。该策略会存储所有 Token 的完整 SWA KV 条目，从而确保计算零冗余。在该策略下，只需读取命中前缀中最后 n_{win} 个 Token 的磁盘缓存，就可以重建该前缀的 SWA KV 条目。尽管计算零冗余，这种策略对于现代基于 SSD 的存储系统并不高效——每个命中请求只会访问已存储 SWA KV cache 的一小部分，导致读写模式严重失衡且偏写入。
- 周期性检查点。该策略会每隔 p 个 Token 对最后 n_{win} 个 Token 的 SWA KV 条目做一次检查点保存，其中 p 是可调参数。对于命中前缀，我们会加载最近一次检查点状态，然后重新计算剩余尾部 Token。通过调节 p ，该策略能够按需在存储与计算之间进行权衡。
- 零 SWA 缓存。该策略不存储任何 SWA KV 条目。对于命中前缀，我们需要进行更多重计算来恢复 SWA KV 条目。具体而言，在每个注意力层中，每个 Token 的 SWA KV 条目只依赖于前一层中最近 n_{win} 个 Token 的 SWA KV 条目。因此，借助已缓存的 CSA 和 HCA KV 条目，只需重计算最后 $n_{\text{win}} \cdot L$ 个 Token，就足以为一个具有 L 层的模型恢复最后 n_{win} 个 SWA KV 条目。

我们会根据具体部署场景选择最合适的策略，以实现期望的存储—计算权衡。

4. 预训练

4.1. 数据构建

在 DeepSeek-V3 的预训练数据基础上，我们致力于构建一个更加多样化、质量更高、且具有更长有效上下文的训练语料。我们持续改进数据构建流水线。对于来源于网页的数据，我们实施了过滤策略，以去除批量自动生成和模板化内容，从而降低模型坍塌风险 (Zhu et al., 2024)。数学与编程语料仍是训练数据的核心组成部分，我们还在中期训练阶段引入智能体数据，进一步增强 DeepSeek-V4 系列的编码能力。对于多语言数据，我们为 DeepSeek-V4 构建了更大的语料，以改善其对不同文化中长尾知识的捕捉能力。对于 DeepSeek-V4，我们特别强调长文档数据整理，优先纳入科学论文、技术报告以及其他体现独特学术价值的材料。综合上述各类数据，我们的预训练语料包含超过 32T 个 Token，涵盖数学内容、代码、网页、长文档以及其他高质量类别。

对于预训练数据，我们大体沿用了与 DeepSeek-

原文第 25 页

V3 相同的预处理策略。在分词方面，我们在 DeepSeek-V3 tokenizer 的基础上引入了少量用于上下文构造的特殊 Token，同时仍将词表大小保持在 128K。我们还继承了 DeepSeek-V3 的 token-splitting (DeepSeek-AI, 2024) 和 Fill-in-Middle (FIM) (DeepSeek-AI, 2024) 策略。受 Ding et al. (2024) 启发，我们将来自不同来源的文档打包成合适的序列，以尽量减少样本截断。与 DeepSeek-V3 不同的是，我们在预训练期间采用了样本级注意力掩码。

4.2. 预训练设置

4.2.1. 模型设置

DeepSeek-V4-Flash。我们将 Transformer 层数设为 43，隐藏维度 d 设为 4096。前两层使用纯滑动窗口注意力。后续各层中，CSA 与 HCA 交替使用。对于 CSA，我们将压缩率 m 设为 4，indexer 查询头数 nlh 设为 64，indexer 头维度 cl 设为 128，用于稀疏注意力所选 KV 条目数（即 attention top-k）设为 512。对于 HCA，我们将压缩率 m' 设为 128。对于 CSA 与 HCA，我们都将查询头数 nh 设为 64，头维度 c 设为 512，查询压缩维度 dc 设为 1024。输出投影分组数 g 设为 8，每个中间注意力输出的维度 dg 设为 1024。对于滑动窗口注意力的附加分支，窗口大小 $nwin$ 设为 128。我们在所有 Transformer 块中都使用 MoE 层，但前 3 个 MoE 层采用 Hash 路由策略。每个 MoE 层包含 1 个共享专家和 256 个路由专家，每个专家的中间隐藏维度为 2048。在这些路由专家中，每个 Token 会激活 6 个专家。多 Token 预测深度设为 1。对于 mHC，扩展因子 nhc 设为 4，Sinkhorn-Knopp 迭代次数 $tmax$ 设为 20。在该配置下，DeepSeek-V4-Flash 总参数量为 284B，其中每个 Token 激活 13B 参数。

DeepSeek-V4-Pro。我们将 Transformer 层数设为 61，隐藏维度 d 设为 7168。前两层使用 HCA。后续各层中，CSA 与 HCA 交替使用。对于 CSA，我们将压缩率 m 设为 4，indexer 查询头数 n_{lh} 设为 64，indexer 头维度 cl 设为 128，用于稀疏注意力所选 KV 条目数（即 attention top-k）设为 1024。对于 HCA，我们将压缩率 m' 设为 128。对于 CSA 与 HCA，我们都将查询头数 nh 设为 128，头维度 c 设为 512，查询压缩维度 dc 设为 1536。输出投影分组数 g 设为 16，每个中间注意力输出的维度 dg 设为 1024。对于滑动窗口注意力的附加分支，窗口大小 n_{win} 设为 128。我们在所有 Transformer 块中都使用 MoE 层，但前 3 个 MoE 层采用 Hash 路由策略。每个 MoE 层包含 1 个共享专家和 384 个路由专家，每个专家的中间隐藏维度为 3072。在这些路由专家中，每个 Token 会激活 6 个专家。多 Token 预测深度设为 1。对于 mHC，扩展因子 n_{hc} 设为 4，Sinkhorn-Knopp 迭代次数 t_{max} 设为 20。在该配置下，DeepSeek-V4-Pro 总参数量为 1.6T，其中每个 Token 激活 49B 参数。

4.2.2. 训练设置

DeepSeek-V4-Flash。对于绝大多数参数，我们采用 Muon 优化器 (Jordan et al., 2024; Liu et al., 2025)，但对

原文第 26 页

嵌入模块、预测头模块以及所有 RMSNorm 模块的权重使用 AdamW 优化器 (Loshchilov and Hutter, 2017)。对于 AdamW，我们将其超参数设置为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.95$ 、 $\epsilon = 10^{-20}$ ，以及 $weight_decay = 0.1$ 。对于 Muon，我们将动量设为 0.95、权重衰减设为 0.1，并将每个更新矩阵的 RMS 重缩放到 0.18，以复用 AdamW 学习率。我们在 32T Token 上训练 DeepSeek-V4-Flash，并与 DeepSeek-V3 一样采用 batch size 调度策略：先将 batch size（以 Token 计）从较小值逐步增加到 75.5M，再在大部分训练过程中保持在 75.5M。学习率在前 2000 步线性预热，在训练的大部分阶段维持为 2.7×10^{-4} 。接近训练末期时，再按照余弦调度将学习率衰减到 2.7×10^{-5} 。训练从 4K 序列长度开始，并逐步扩展到 16K、64K 和 1M。对于稀疏注意力设置，我们在前 1T Token 中先使用稠密注意力对模型进行预热，在序列长度达到 64K 时引入稀疏注意力，并在余下训练过程中持续使用稀疏注意力。引入注意力稀疏性时，我们先设置一个短阶段来预热 CSA 中的 lightning indexer，随后在大部分训练中使用稀疏注意力训练模型。对于无辅助损失的负载均衡，我们将偏置更新速度设为 0.001。对于 balance loss，我们将其权重设为 0.0001，以避免单个序列内部的极端失衡。MTP 损失权重在大部分训练期间设为 0.3，在学习率开始衰减时设为 0.1。

DeepSeek-V4-Pro。除某些超参数的具体取值外，DeepSeek-V4-Pro 的训练设置与 DeepSeek-V4-Flash 基本一致。我们对绝大多数参数使用 Muon 优化器，但对嵌入模块、预测头模块以及所有 RMSNorm 模块的权重使用 AdamW 优化器。AdamW 和 Muon 的超参数与 DeepSeek-V4-Flash 相同。我们在 33T Token 上训练 DeepSeek-V4-Pro，同样采用 batch

size 调度策略，最大 batch size 为 94.4M Token。学习率调度策略与 DeepSeek-V4-Flash 基本相同，但峰值学习率设为 2.0×10^{-4} ，最终学习率设为 2.0×10^{-5} 。训练同样从 4K 序列长度开始，并逐步扩展到 16K、64K 和 1M。与 DeepSeek-V4-Flash 相比，DeepSeek-V4-Pro 的稠密注意力阶段更长，而引入稀疏注意力的策略与 DeepSeek-V4-Flash 相同，遵循两阶段训练方法。对于无辅助损失的负载均衡，我们将偏置更新速度设为 0.001。对于 balance loss，我们将其权重设为 0.0001，以避免单个序列内部的极端失衡。MTP 损失权重在大部分训练期间设为 0.3，在学习率开始衰减时设为 0.1。

4.2.3. 缓解训练不稳定性

训练万亿参数级的 MoE 模型会带来显著的稳定性挑战，DeepSeek-V4 系列也不例外。我们在训练过程中遭遇了明显的不稳定问题。虽然简单回滚可以暂时恢复训练状态，但它并不是长期解决方案，因为它无法阻止 loss spike 的再次发生。根据经验，我们发现 spike 的出现始终与 MoE 层中的异常值有关，而路由机制本身似乎又会加剧这些异常值的产生。因此，我们尝试从两个维度解决这一问题：一是打破路由诱发的恶性循环，二是直接抑制异常值。幸运的是，我们发现了两种能够有效维持训练稳定性的实用技术。尽管目前对其底层机制的完整理论理解仍是一个开放问题，但我们仍将其公开分享，以推动社区进一步探索。

原文第 27 页

预先路由 (Anticipatory Routing)。我们发现，将骨干网络与路由网络的同步更新解耦，能够显著提升训练稳定性。因此，在步骤 t ，我们使用当前网络参数 θ_t 来计算特征，但路由索引则使用历史网络参数 $\theta_{t-\Delta t}$ 来计算并应用。在实践中，为了避免两次加载模型参数的开销，我们会在步骤 $t-\Delta t$ 时提前抓取步骤 t 所需的数据。我们“预先”计算并缓存稍后在步骤 t 使用的路由索引，因此将这种方法命名为 Anticipatory Routing。我们还在基础设施层面对其做了大量优化。首先，由于预计算路由索引只需要对数据执行一次前向传播，我们精心编排了流水线执行以及计算与专家并行 (EP) 通信的重叠，成功将 Anticipatory Routing 的额外 wall-clock 时间开销控制在大约 20%。其次，我们引入了自动检测机制：仅在出现 loss spike 时触发一次短回滚并激活 Anticipatory Routing；在该模式运行一段时间后，系统会恢复到标准训练。最终，这种动态应用方式使我们能够以几乎可忽略的整体额外训练开销规避 loss spike，同时又不损害模型性能。

SwiGLU 截断 (Clamping)。在以往文献中 (Bello et al., 2017; Riviere et al., 2024)，clamping 已被明确用于约束数值范围，从而提升训练稳定性。在我们的实际训练中，我们通过经验发现，应用 SwiGLU clamping (OpenAI, 2025) 能够有效消除异常值，并在不损害性能的情况下显著帮助稳定训练过程。在 DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 的整个训练过程中，我们将 SwiGLU 的线性分量截断在 $[-10, 10]$ 范围内，同时将 gate 分量的上界限制为 10。

4.3. 评测

4.3.1. 评测基准

在基座模型评测中，我们考虑覆盖四个关键维度的基准：世界知识、语言理解与推理、代码与数学，以及长上下文处理。

世界知识类基准包括 AGIEval (Zhong et al., 2023)、C-Eval (Huang et al., 2023)、CMMLU (Li et al., 2023)、MMLU (Hendrycks et al., 2020)、MMLU-Redux (Gema et al., 2024)、MMLU-Pro (Wang et al., 2024b)、MMMLU (OpenAI, 2024a)、MultiLoKo (Hupkes and Bogoychev, 2025)、Simple-QA verified (Haas et al., 2025)、SuperGPQA (Du et al., 2025)、FACTS Parametric (Cheng et al., 2025) 以及 TriviaQA (Joshi et al., 2017)。

语言理解与推理类基准包括 BigBench Hard (BBH) (Suzgun et al., 2022)、DROP (Dua et al., 2019)、HellaSwag (Zellers et al., 2019)、CLUEWSC (Xu et al., 2020) 以及 WinoGrande (Sakaguchi et al., 2019)。

代码与数学类基准包括 BigCodeBench (Zhuo et al., 2025)、HumanEval (Chen et al., 2021)、GSM8K (Cobbe et al., 2021)、MATH (Hendrycks et al., 2021)、MGSM (Shi et al., 2023) 以及 CMath (Wei et al., 2023)。

长上下文类基准包括 LongBench-V2 (Bai et al., 2025b)。

基准 (指标) # ShotsDeepSeek-V3.2DeepSeek-V4-FlashDeepSeek-V4-ProBaseBaseBase

架构-MoEMoEMoE# 激活参数-37B13B49B# 总参数-671B284B1.6T

World Knowl.	AGIEval (EM)	0-shot	80.1	<u>82.6</u>	83.1
	MMLU (EM)	5-shot	87.8	<u>88.7</u>	90.1
	MMLU-Redux (EM)	5-shot	87.5	<u>89.4</u>	90.8
	MMLU-Pro (EM)	5-shot	65.5	<u>68.3</u>	73.5
	MMMLU (EM)	5-shot	87.9	<u>88.8</u>	90.3
	C-Eval (EM)	5-shot	90.4	<u>92.1</u>	93.1
	CMMLU (EM)	5-shot	88.9	<u>90.4</u>	90.8
	MultiLoKo (EM)	5-shot	38.7	<u>42.2</u>	51.1
	Simple-QA verified (EM)	25-shot	28.3	<u>30.1</u>	55.2
	SuperGPQA (EM)	5-shot	45.0	<u>46.5</u>	53.9
	FACTS Parametric (EM)	25-shot	27.1	<u>33.9</u>	62.6
	TriviaQA (EM)	5-shot	<u>83.3</u>	82.8	85.6
Lang. & Reas.	BBH (EM)	3-shot	87.6	<u>86.9</u>	87.5
	DROP (F1)	1-shot	<u>88.2</u>	88.6	88.7
	HellaSwag (EM)	0-shot	<u>86.4</u>	85.7	88.0
	WinoGrande (EM)	0-shot	<u>78.9</u>	<u>79.5</u>	81.5
	CLUEWSC (EM)	5-shot	<u>83.5</u>	<u>82.2</u>	85.2
Code & Math	BigCodeBench (Pass@1)	3-shot	63.9	56.8	<u>59.2</u>
	HumanEval (Pass@1)	0-shot	62.8	<u>69.5</u>	76.8
	GSM8K (EM)	8-shot	<u>91.1</u>	90.8	92.6
	MATH (EM)	4-shot	<u>60.5</u>	57.4	64.5
	MGSM (EM)	8-shot	81.3	85.7	<u>84.4</u>
	CMath (EM)	3-shot	<u>92.6</u>	93.6	90.9

表 1 | DeepSeek-V3.2-Base、DeepSeek-V4-Flash-Base 与 DeepSeek-V4-Pro-Base 的对比。所有模型都在我们的内部评测框架下、使用完全相同的评测设置进行评估。分差不超过 0.3 的结果视为处于同一水平。每一行中的最高分以粗体显示，第二高分以下划线显示。

长上下文LongBench-V2 (EM) 1-shot40.244.751.5

4.3.2. 评测结果

在表 1 中，我们对 DeepSeek-V3.2、DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的基座模型给出了详细对比，所有模型都在统一的内部框架下、以严格一致的设置进行评测。

将 DeepSeek-V4-Flash-Base 与 DeepSeek-V3.2-Base 进行比较，可以看到一个极具说服力的效率故事。尽管 DeepSeek-V4-Flash-Base 使用的激活参数和总参数数量都明显更少，但它在广泛的基准测试中全面超越了 DeepSeek-V3.2-Base。这一优势在世界知识任务和具有挑战性的长上下文场景中尤为明显。这些结果表明，DeepSeek-V4-Flash-Base 的架构改进、数据质量提升以及训练优化，即便在更紧凑的参数预算下，也能带来更优性能，从而在大多数评测中有效超越参数更多的 DeepSeek-V3.2-Base。

此外，DeepSeek-V4-Pro-Base 还展示了进一步且决定性的能力跃升，在 DeepSeek-V3.2-Base 与 DeepSeek-V4-Flash-Base 之上建立了近乎全面的优势。随着几乎所有类别的提升，DeepSeek-V4-Pro-Base 达到了新的

在最具挑战性的基准上，DeepSeek 基座模型的性能达到了新高。在知识密集型评测中，它带来了显著提升，同时也大幅推进了长上下文理解能力。在大多数推理和代码基准上，DeepSeek-V4-Pro-Base 也超过了此前两个模型。这种全面提升确认了 DeepSeek-V4-Pro-Base 是 DeepSeek 系列中最强的基座模型，在知识、推理、编码和长上下文能力等各个维度都优于其前代。

5. 后训练

5.1. 后训练流水线

在预训练之后，我们开展了后训练阶段，以产出 DeepSeek-V4 系列的最终模型。尽管训练流水线在很大程度上与 DeepSeek-V3.2 相似，但其中有一个关键的方法学替换：混合强化学习（RL）阶段被完全替换为 On-Policy Distillation（OPD）。

5.1.1. 专家化训练

领域专家模型的开发是通过改造 DeepSeek-V3.2 的训练流水线来完成的。具体而言，每个模型都依次经过初始微调阶段以及随后由领域特定提示和奖励信号引导的强化学习（RL）阶段。对于 RL 阶段，我们实现了 Group Relative Policy Optimization（GRPO）算法，并使其超参数与我们先前研究（DeepSeek-AI, 2025; DeepSeek-AI, 2025）保持高度一致。

推理强度。众所周知，模型在推理任务上的表现从根本上受其所消耗计算量的支配。因此，我们在不同的 RL 配置下训练了不同的专家模型，以促进形成针对不同推理能力进行优化的模型。正如表 2 所示，DeepSeek-V4-Pro 和 DeepSeek-V4-Flash 都支持三种特定的推理强度模式。对于每种模式，我们在 RL 训练期间采用不同的长度惩罚和上下文窗口，从而得到不同的推理输出 Token 长度。为了整合这些不同的推理模式，我们使用由 `<think>` 和 `</think>` 标记界定的专门响应格式。此外，对于“Think Max”模式，我们会在系统提示开头加入特定指令，以引导模型的推理过程，如表 3 所示。

生成式奖励模型。通常，容易验证的任务可以通过简单的基于规则的验证器或测试用例来有效优化。相比之下，难以验证的任务传统上依赖于基于人类反馈的强化学习（RLHF），这需要大量人工标注来训练一个标量奖励模型。然而，在 DeepSeek-V4 系列的后训练阶段，我们摒弃了这些传统的基于标量的奖励模型。相反，为处理难验证任务，我们构建了 rubric-guided RL 数据，并采用生成式奖励模型（GRM）来评估策略轨迹。关键在于，我们将 RL 优化直接应用于 GRM 本身。在这一范式中，actor 网络天然地充当 GRM，从而使模型的评估（判别）能力与其常规生成能力得以联合优化。通过统一这两种角色，模型内部的推理能力会自然地融入其评估过程中，从而产生高度稳健的打分。此外，这种方法只需极少量多样化的人类标注便能取得更优性能，因为

推理模式特征典型使用场景响应格式

`</think> summary`

Non-think 基于习惯或简单规则的快速、直觉式响应。

日常例行任务、紧急反应、低风险决策。

`<think> thinkingtokens </think> summary`

Think High 有意识的逻辑分析，较慢但更准确。

复杂问题求解、规划、中等风险决策。

Think Max 将推理推到其可能的极限。速度慢，但能力强。

1. 在开头加入一条特殊系统提示。 2. `<think> thinkingtokens </think> summary`

探索模型推理能力的边界。

Injected Instruction

Reasoning Effort: Absolute maximum with no shortcuts permitted.

You MUST be very thorough in your thinking and comprehensively decompose the problem to resolve the root cause, rigorously stress-testing your logic against all potential paths, edge cases, and adversarial scenarios.

Explicitly write out your entire deliberation process, documenting every intermediate step, considered alternative, and rejected hypothesis to ensure absolutely no assumption is left unchecked.

表 3 | 为 “Think Max” 模式注入到系统提示中的指令。

模型会利用自身逻辑在复杂任务之间进行泛化。

工具调用模式与特殊 Token。与我们上一版本一致，我们使用专用的 `<think> </think>` 标签来界定推理路径。在 DeepSeek-V4 系列中，我们引入了一种新的工具调用模式：它使用特殊的 “[DSML]” Token，并采用基于 XML 的格式进行工具调用，如表 4 所示。我们的实验表明，XML 格式能够有效缓解转义失败并减少工具调用错误，从而为模型—工具交互提供更稳健的接口。

交错思考 (Interleaved Thinking) 。DeepSeek-V3.2 引入了一种上下文管理策略：在工具结果往返轮次之间保留推理轨迹，但在新用户消息到来时丢弃这些轨迹。虽然该策略有效，但在复杂智能体工作流中仍会造成不必要的 Token 浪费——每次新用户轮次都会清空所有累积推理内容，迫使模型从头重建其问题求解状态。借助扩展到 1M Token 的上下文

Tool Call Schema

```
## Tools

You have access to a set of tools to help answer the user's question. You can
  invoke tools by writing a "<|DSML|tool_calls>" block like the following:

<|DSML|tool_calls>
<|DSML|invoke name="$TOOL_NAME">
<|DSML|parameter name="$PARAMETER_NAME" string="true|false">$PARAMETER_VALUE
  </|DSML|parameter>
...
</|DSML|invoke>
<|DSML|invoke name="$TOOL_NAME2">
...
</|DSML|invoke>
</|DSML|tool_calls>

String parameters should be specified as is and set 'string="true"'. For all
  other types (numbers, booleans, arrays, objects), pass the value in JSON
  format and set 'string="false"'.

If thinking_mode is enabled (triggered by <think>), you MUST output your
  complete reasoning inside <think>...</think> BEFORE any tool calls or
  final response.

Otherwise, output directly after </think> with tool calls or final response.

### Available Tool Schemas

{Tool Definition...}

You MUST strictly follow the above definedtool name and parameter schemas to
  invoke tool calls.
```

表 4 | DeepSeek-V4 系列的工具调用模式。

窗口，DeepSeek-V4 系列进一步完善了这一机制，以在智能体环境中最大化交错思考的效果：

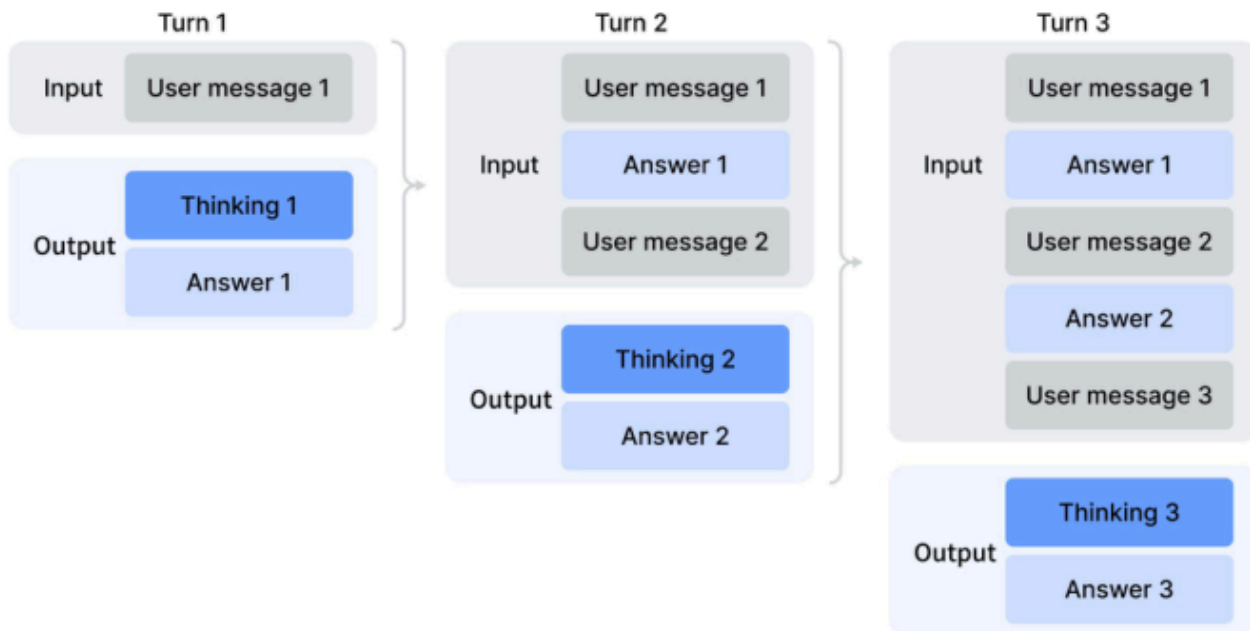
- 工具调用场景。如图 7(a) 所示，所有推理内容都会在整个对话过程中被完整保留。与 DeepSeek-V3.2 在每次新用户轮次到来时丢弃思考轨迹不同，DeepSeek-V4 系列会在所有轮次中保留完整的推理历史，包括跨越用户消息边界的情况。这使模型能够在长时程智能体任务中维持连贯、累积的思维链。
- 一般对话场景。如图 7(b) 所示，原始策略仍被保留：当新用户消息到

来时，会丢弃前几轮中的推理内容，从而在那些持久推理轨迹收益有限的场景中保持上下文简洁。

与 DeepSeek-V3.2 一样，那些通过用户消息模拟工具交互的智能体框架（例如 Terminus）可能不会触发工具调用上下文路径，因此也可能无法从增强的推理持久化中受益。对于这种架构，我们仍然建议使用非思考模型。



a) Thinking with tools



b) Thinking without tools

Quick Instruction. 在聊天机器人场景中，在生成回复之前，必须先执行若干辅助任务（例如判断是否触发网页搜索、意图识别等）。传统上，这些任务由一个独立的小模型处理，而由于它无法复用现有 KV cache，因此需要冗余的 prefilling。为了克服这一限制，我们引入了 Quick

Instruction。我们直接在输入序列末尾附加一组专用特殊 Token，其中每个 Token 都对应一个特定的辅助任务。通过直接复用已计算出的 KV cache，这一机制完全避免了冗余 prefilling，并允许某些任务（例如生成搜索查询、判断权威性和领域）并行执行。因此，这种方法显著降低了用户感知的首 Token 时间（TTFT），也消除了维护和迭代额外小模型的工程负担。支持的 Quick Instruction Token 总结见表 5。

5.1.2. On-Policy Distillation

在通过专门微调与强化学习训练多个领域专家后，我们采用多教师 On-Policy Distillation（OPD）作为将专家能力合并到最终模型中的主要技术。OPD 已成为一种有效的后训练范式，能够高效地将领域专家的知识 and 能力迁移到单一统一模型中。其实现方式是让学生模型在自己生成的轨迹上学习教师模型的输出分布。形式化地，给定一组 N 个专家模型

图 7 | DeepSeek-V4 系列的思考管理方式。

原文第 33 页

特殊 Token描述格式

<|action|>判断用户提示是否需要网页搜索还是可以直接回答。

...<|User|>{prompt}<|Assistant|><think><|action|>

<|title|>在第一次助手回复后生成简洁的对话标题。

...<|Assistant|>{response}<|end_of_sentence|><|title|>

<|query|>为用户提示生成搜索查询。...<|User|>{prompt}<|query|>

<|authority|>对用户提示所需的来源权威性进行分类。

...<|User|>{prompt}<|authority|>

<|domain|>识别用户提示所属领域。...<|User|>{prompt}<|domain|>

<|extracted_url|><|read_url|>判断用户提示中的每个 URL是否应被抓取并读取。

...<|User|>{prompt}<|extracted_url|>{url}<|read_url|>

$\{\pi E1, \pi E2, \dots, \pi EN\}$, OPD 的目标函数定义为：

$$\sum_{i=1}^N w_i \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i}). \quad (29)$$

$$i = 1 w_i \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i}). \quad (29)$$

LOPD(θ) =

在这一表述中， w_i 表示分配给每个专家的权重，通常由该专家的相对重要性决定。计算反向 KL 损失 $D_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i})$ 需要从学生模型 π_{θ} 中采样训练轨迹，以维持 on-policy 学习。其底层逻辑确保统一策略 π_{θ} 会有选择地从与当前任务上下文相关的专门专家那里学习（例如在数学推理任务中对齐数学专家，在编程任务中对齐代码专家）。通过这一机制，来自物理上彼此独立的专家权重中的知识会通过 logit 级别对齐被整合到统一参数空间中，从而在实践中规避传统权重合并或混合 RL 技术中经常出现的性能退化。在这一阶段，我们使用了十多个覆盖不同领域的教师模型来蒸馏单一学生模型。

在处理上述 OPD 目标时，以往工作通常会将完整词表 KL 损失简化为每个 Token 位置上的 token-level KL 估计，并通过将 $\text{sg}[\log(\pi_{E_i}(y_t|x, y < t))$

$/ \pi_{\theta}(y_t|x, y < t)]$ （其中 sg 表示 stop gradient 操作）替换为策略损失计算中的逐 Token advantage 估计，来复用 RL 框架。尽管这种方法资源效率较高，但它会导致梯度估计方差较大，并常常引发训练不稳定。因此，我们在 OPD 中采用完整词表 logit 蒸馏。在计算反向 KL 损失时保留完整的 logit 分布，可以得到更稳定的梯度估计，并确保对教师知识的忠实蒸馏。在下一个小节中，我们将介绍使大规模完整词表 OPD 成为可能的工程工作。

表 5 | 用于辅助任务的 Quick Instruction 特殊 Token。

原文第 34 页

5.2. RL 与 OPD 基础设施

我们的后训练基础设施建立在为 DeepSeek-V3.2 开发的可扩展框架之上。具体来说，我们集成了第 3.5 节所述的同一套分布式训练栈，以及此前引入的用于高效自回归采样的 rollout 引擎。在此基础上，我们在本工作中引入了以下主要增强。这些设计能够高效执行涉及十多个不同教师模型的超长上下文 RL 与 OPD 合并任务，从而显著加快模型发布的迭代周期。

5.2.1. FP4 量化集成

我们应用 FP4 (MXFP4) 量化来加速 rollout 以及所有仅推理的前向过程，包括教师模型与参考模型的前向过程，从而减少内存流量并降低采样延迟。如第 3.4 节所述，在 rollout 和推理阶

段，我们直接使用原生 FP4 权重。对于训练步骤，则通过一次无损的 FP4 到 FP8 反量化来模拟 FP4 量化，从而能够无缝复用现有的、使用 FP32 主权重的 FP8 混合精度框架，并且无需修改反向传播流水线。

5.2.2. 面向完整词表 OPD 的高效教师调度

我们的框架支持完整词表 On-Policy Distillation (OPD)，教师数量在效果上不设上限，而每个教师模型都可能拥有万亿级参数。为实现这一点，所有教师权重都会卸载到集中式分布式存储中，并在教师前向传播期间按需加载，同时采用类似 ZeRO 的参数分片来缓解 I/O 与 DRAM 压力。此外，即便将 logits 临时写到磁盘上，若要为所有教师实体化词表大小 $|V| > 100k$ 的 logits，仍然是不可承受的。为此，我们在前向过程中仅将最后一层教师隐藏状态缓存到一个中心缓冲区中。训练时，这些缓存状态会被取回，并通过对应的预测头模块在线重建完整 logits。该设计重计算开销几乎可以忽略不计，却完全规避了显式实体化 logits 所带来的内存负担。为缓解教师预测头的 GPU 内存占用，我们在数据分发时按教师索引对训练样本进行排序。这样可确保每个不同教师头在每个 mini-batch 中只需加载一次，并且任意时刻设备内存中最多只驻留一个教师头。所有参数和隐藏状态的加载/卸载操作都在后台异步进行，不会阻塞关键路径上的计算。最后，教师与学生 logits 之间的精确 KL 散度由一个专门的 TileLang 内核来计算，该内核加速了计算并抑制了动态内存分配。

5.2.3. 可抢占且具容错能力的 Rollout 服务

为了最大化 GPU 资源利用率，同时支持为高优先级任务快速调配硬件资源，我们的 GPU 集群采用了集群级可抢占任务调度器，任何正在运行的任务都可能在任意时刻被抢占。此外，在大规模 GPU 集群中，硬件故障也很常见。为此，我们为 RL/OPD rollout 实现了一套可抢占且具容错能力的大语言模型生成服务。

具体来说，我们为每个生成请求实现了一个按 Token 粒度的预写日志 (WAL)。每当某个请求生成一个新 Token，我们都会立即将其追加到该请求的 WAL 中。在发生抢占时，我们会暂停推理引擎，并保存 KV cache 的

原文第 35 页

未完成请求状态。恢复时，我们利用持久化的 WAL 和已保存的 KV cache 继续解码。即使发生致命硬件错误，我们也可以利用 WAL 中持久化的 Token 重新运行 prefill 阶段，以重建 KV cache。

重要的是，从头重新生成未完成请求在数学上是不正确的，因为这会引入长度偏差。由于较短的回复更容易在中断中幸存，如果每次中断后都从头重生成，模型就会更倾向于生成更短序列。如果推理栈具备批不变性和确定性，那么这一正确性问题也可以通过使用一致的采样器伪随机数生

成器种子来重生成加以解决。然而，这种方式仍然需要额外重跑解码阶段，因此远不如我们的按 Token 粒度 WAL 方法高效。

5.2.4. 面向百万 Token 上下文的可扩展 RL 框架

我们引入了一系列针对性优化，以支持在百万 Token 序列上高效执行 RL 和 OPD。在 rollout 阶段，我们采用了第 5.2.3 节详细介绍的可抢占、可容错 rollout 服务。对于推理与训练阶段，我们将 rollout 数据格式拆分为轻量元数据和重量级逐 Token 字段。在数据分发期间，可以先加载整个 rollout 数据的元数据，以进行全局洗牌和打包布局计算。重量级逐 Token 字段则通过共享内存数据加载器进行加载，以消除节点内数据冗余，并在 mini-batch 粒度上于消费后立即释放，从而大幅降低 CPU 与 GPU 内存压力。设备上 mini-batch 的数量会根据工作负载动态确定，从而在计算吞吐与 I/O 重叠之间实现高效权衡。

5.2.5. 面向智能体 AI 的沙箱基础设施

为了满足智能体 AI 在后训练与评估阶段多样化的执行需求，我们构建了一套生产级沙箱平台 DeepSeek Elastic Compute (DSec)。DSec 由三个 Rust 组件构成——API 网关 (Apiserver)、每主机代理 (Edge) 以及集群监控器 (Watcher)——它们通过自定义 RPC 协议互联，并构建在 3FS 分布式文件系统 (DeepSeek-AI, 2025) 之上以实现水平扩展。在生产环境中，单个 DSec 集群可管理数十万个并发沙箱实例。

DSec 的设计基于四点观察：（1）智能体工作负载高度异构，从轻量级函数调用到完整的软件工程流水线不等，且具有不同的操作系统与安全需求；（2）环境镜像数量多且体积大，但必须快速加载并支持迭代式定制；（3）高密度部署要求高效利用 CPU 和内存；（4）沙箱生命周期必须与 GPU 训练调度（包括抢占与基于检查点的恢复）协同。基于这些观察，我们将在下文分别介绍 DSec 的四项核心设计。

统一接口背后的四种执行基底。DSec 提供单一 Python SDK (libdsec)，抽象了四种执行基底。Function Call 会将无状态调用分发到预热容器池中，从而消除冷启动开销。Container 完全兼容 Docker，并利用 EROFS (Gao et al., 2019) 的按需加载能力高效构建镜像。microVM 基于 Firecracker (Agache et al., 2020)，为安全敏感且高密度部署场景提供虚拟机级隔离。fullVM 基于 QEMU (Bellard, 2005)，支持任意访客操作系统。这四种基底共享同一套 API 表面——命令执行、

文件传输和 TTY 访问——而在它们之间切换只需修改一个参数。

通过分层存储实现快速镜像加载。DSec 通过分层、按需加载方式，在快速启动和庞大且持续增长的环境镜像语料之间取得平衡。对于容器，基础镜像和文件系统提交以 3FS 支持的只读

EROFS 层形式存储，并直接挂载到 overlay lowerdirs 中。我们在挂载时就将文件元数据准备在本地磁盘上；与此同时，数据块会按需从 3FS 拉取。对于 microVM，DSec 使用 overlaybd (Li et al., 2020) 磁盘格式：只读基础层驻留在 3FS 上以供跨实例共享，而写入则进入本地 copy-on-write 层。这类快照可以进行链式组织，从而支持高效版本管理与毫秒级恢复。

大规模并发下的密度优化。为在每个集群中容纳数十万个沙箱，DSec 针对两个资源瓶颈进行了优化。第一，它减轻了虚拟化环境中重复页缓存带来的内存占用，并采用内存回收机制以支持安全超额分配。第二，它缓解了容器运行时中的 spinlock 竞争，从而降低了单沙箱的 CPU 开销，并显著提高了每台主机的打包密度。

轨迹日志与支持抢占安全恢复。DSec 为每个沙箱维护一份全局有序的轨迹日志，持久记录每次命令调用及其结果。该轨迹有三个用途：（1）客户端快进——当训练任务被抢占时，沙箱资源仍会被保留；恢复后，DSec 会重放先前已完成命令的缓存结果，从而加速任务恢复，并同时避免对非幂等操作重复执行所导致的错误；（2）细粒度溯源——每一次状态变化的来源及其对应结果都可追踪；（3）确定性回放——任意历史会话都可以依据其轨迹被忠实复现。

5.3. 标准基准评测

5.3.1. 评测设置

知识与推理。知识与推理数据集包括 MMLU-Pro (Wang et al., 2024b)、GPQA (Rein et al., 2023)、Human Last Exam (Phan et al., 2025)、Simple-QA Verified (Haas et al., 2025)、Chinese-SimpleQA (He et al., 2024)、LiveCodeBench-v6 (Jain et al., 2024)、CodeForces (内部基准)、HMMT 2026 Feb、Apex (Balunović et al., 2025)、Apex Shortlist (Balunović et al., 2025)、IMOAnswerBench (Luong et al., 2025) 以及 PutnamBench (Tsoukalas et al., 2024)。

对于代码，我们在 LiveCodeBench-v6 和一个内部 Codeforces 基准上评测 DeepSeek-V4 系列。对于 Codeforces，我们收集了 14 场 Codeforces Division 1 比赛，共 114 道题（2025 年 5 月至 2025 年 11 月）。Elo 评分按如下方式计算。对于每场比赛，我们为每道题生成 32 个候选解。对于每道题，我们独立地从这 32 个解中无放回采样 10 个，并将其随机排序形成提交序列。每次提交都会由领域专家构建的测试集进行评判。已解出题目的得分遵循 OpenAI (2025) 的惩罚方案：模型会获得那些以相同先前失败次数解出同题的人类参赛者的中位数得分。由此，我们可为每个采样得到的提交序列计算出总比赛得分，并将其转换为比赛名次，进一步换算为估计 rating，所用的是标准 Codeforces 评级系统。比赛级别的期望 rating 被定义为

该估计 rating 在所有可能的、每题 10 个提交的随机选择与排序上的期望。模型的总体 rating 则是这 14 场比赛的比赛级别期望 rating 的平均值。

对于推理与知识任务，我们将温度设为 1.0，并分别为 Non-think、High 和 Max 模式设置 8K、128K 和 384K Token 的上下文窗口。对于数学任务（例如 HMMT、IMOAnswerBench、Apex 和 HLE），我们采用如下模板进行评测：“{question}Please reason step by step, and put your final answer within\boxed{.}”对于数学任务上的 DeepSeek-V4-Pro-Max，我们使用如下模板来诱导更深层推理：“Solve the following problem.The problem may ask you to prove a statement, or ask for an answer.If finding an answer is required,you should come up with the answer, and your final solution should also be a rigorous proof of that answer being valid.{question}”。

对于形式化数学任务，我们在智能体设置中、基于 Lean v4.28.0-rc1（Moura and Ullrich, 2021）进行评测，并允许访问 Lean 编译器和语义 tactic 搜索引擎，在最大推理强度下最多运行 500 次工具调用。此外，我们还评测了一种计算量更大的流程：先生成候选自然语言解答，并通过自我验证（Shao et al., 2025）进行筛选，再将保留下来的解答作为指导提供给形式化智能体，以证明相应的 Lean 命题。这种设计使用非形式推理来改善探索，同时通过形式化验证保持严格正确性。只有当严格验证器 Comparator 在这两种设置下都接受某一提交时，我们才将其记为正确。

对于 K2.6 和 GLM-5.1，我们保留了一些空白条目，因为它们 API 过于繁忙，无法对我们的查询返回响应。

1M-Token 上下文。由于 DeepSeek-V4 系列支持 1M-Token 上下文，我们通过选择 OpenAI MRCR（OpenAI, 2024b）和 CorpusQA（Lu et al., 2026）作为基准，来评估模型在长上下文场景中的表现。我们重新评测了 Claude Opus 4.6 和 Gemini 3.1 Pro 在这些任务上的结果，以期在所有模型之间统一配置。我们没有评测 GPT-5.4，因为其 API 对我们的大量查询都未返回响应。

智能体。智能体数据集包括 Terminal Bench 2.0（Merrill et al., 2026）、SWE-Verified（OpenAI, 2024e）、SWE Multilingual（Yang et al., 2025）、SWE-Pro（Deng et al., 2025）、BrowseComp（Wei et al., 2025）、MCPAtlas 的公开评测集（Bandi et al., 2026）、GDPval-AA（AA, 2025; Patwardhan et al., 2025）以及 Tool-Decathlon（Li et al., 2025）。

对于代码智能体任务（SWE-Verified、Terminal-Bench、SWE-Pro、SWE Multilingual），我们使用内部开发的评测框架来评估 DeepSeek-V4 系列。该框架提供一组最小化工具——bash 工具和文件编辑工具。最大交互步数设为 500，最大上下文长度设为 512K Token。关于 Terminal-Bench 2.0，我们注意到 GLM-5.1 所指出的环境相关问题。尽管如此，为保持一致

性，我们仍报告在原始 Terminal-Bench 2.0 数据集上的性能。在 Terminal-Bench 2.0 Verified 子集上，DeepSeek-V4-Pro 取得了约 72.0 的得分。

对于搜索智能体任务（BrowseComp、带工具的 HLE），我们同样使用内部 harness，并配备 websearch 和 Python 工具，同时将最大交互步数设为 500、最大上下文长度设为 512K Token。对于 BrowseComp，我们采用与 DeepSeek-V3.2（DeepSeek-AI, 2025）相同的“丢弃全部上下文”管理策略。

5.3.2. 评测结果

基 准 （ 指 标 ） Opus-4.6 GPT-5.4 Gemini-3.1-ProK2.6GLM-5.1 DS-V4-ProMaxxHighHighThinking ThinkingMax

Knowledge & Reasoning	MMLU-Pro (EM)	<u>89.1</u>	87.5	91.0	87.1	86.0	87.5
	SimpleQA-Verified (Pass@1)	46.2	45.3	75.6	36.9	38.1	<u>57.9</u>
	Chinese-SimpleQA (Pass@1)	76.4	76.8	85.9	75.9	75.0	<u>84.4</u>
	GPQA Diamond (Pass@1)	91.3	<u>93.0</u>	94.3	90.5	86.2	90.1
	HLE (Pass@1)	<u>40.0</u>	39.8	44.4	36.4	34.7	37.7
	LiveCodeBench (Pass@1)	88.8	-	<u>91.7</u>	89.6	-	93.5
	Codeforces (Rating)	-	<u>3168</u>	3052	-	-	3206
	HMMT 2026 Feb (Pass@1)	<u>96.2</u>	97.7	94.7	92.7	89.4	95.2
	IMOAnswerBench (Pass@1)	75.3	91.4	81.0	86.0	83.8	<u>89.8</u>
	Apex (Pass@1)	34.5	<u>54.1</u>	60.9	24.0	11.5	38.3
	Apex Shortlist (Pass@1)	85.9	78.1	<u>89.1</u>	75.5	72.4	90.2

表 6 | DeepSeek-V4-Pro-Max 与闭源/开源模型的对比。“Max”、“xHigh”和“High”表示推理强度。最佳结果以粗体标出，第二佳结果以下划线标出。

长

MRCR 1M (MMR) 92.9-76.3--83.5CorpusQA 1M (ACC) 71.7-53.8--62.0

Agentic	Terminal Bench 2.0 (Acc)	65.4	75.1	<u>68.5</u>	66.7	63.5	67.9
	SWE Verified (Resolved)	80.8	-	<u>80.6</u>	80.2	-	<u>80.6</u>
	SWE Pro (Resolved)	57.3	57.7	<u>54.2</u>	58.6	<u>58.4</u>	55.4
	SWE Multilingual (Resolved)	77.5	-	-	<u>76.7</u>	73.3	<u>76.2</u>
	BrowseComp (Pass@1)	<u>83.7</u>	82.7	85.9	83.2	79.3	83.4
	HLE w/ tools (Pass@1)	<u>53.1</u>	52.0	51.6	54.0	50.4	48.2
	GDPval-AA (Elo)	<u>1619</u>	1674	1314	1482	1535	1554
	MCPAtlas Public(Pass@1)	73.8	67.2	69.2	66.6	71.8	<u>73.6</u>
	Toolathlon (Pass@1)	47.2	54.6	48.8	50.0	40.7	<u>51.8</u>

DeepSeek-V4-Pro-Max 与其他闭源/开源模型的对比见表 6。此外，我们还评测了 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的不同模式，并在表 7 中展示结果。

知识。在通用世界知识评测中，DeepSeek-V4-Pro-Max 作为 DeepSeek-V4-Pro 的最大推理强度模式，在开源大语言模型中树立了新的最先进水平。正如 SimpleQA-Verified 所示，DeepSeek-V4-Pro-Max 以 20 个绝对百分点的优势显著超过所有现有开源基线。尽管取得这些进展，它目前仍落后于领先的专有模型 Gemini-3.1-Pro。在教育知识与推理领域，DeepSeek-V4-Pro-Max 在 MMLU-Pro、GPQA 和 HLE 基准上略微领先于 Kimi 和 GLM，但仍落后于领先的专有模型。总体而言，DeepSeek-V4-Pro-Max 在提升开源模型的世界知识能力方面标志着一个重要里程碑。

此外，在知识类任务上，DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 之间存在显著性能差距；这在预期之中，因为更大的参数规模有利于在预训练期间保留更多知识。值得注意的是，当分配更高的推理强度时，这两个模型在知识类基准上都表现出更好的结果。

基准 (指标) DeepSeek-V4-FlashDeepSeek-V4-ProNon-ThinkHighMaxNon-ThinkHighMax

Knowledge & Reasoning	MMLU-Pro (EM)	83.0	86.4	86.2	82.9	87.1	87.5
	SimpleQA-Verified (Pass@1)	23.1	28.9	34.1	45.0	46.2	57.9
	Chinese-SimpleQA (Pass@1)	71.5	73.2	78.9	75.8	77.7	84.4
	GPQA Diamond (Pass@1)	71.2	87.4	88.1	72.9	89.1	90.1
	HLE (Pass@1)	8.1	29.4	34.8	7.7	34.5	37.7
	LiveCodeBench (Pass@1-COT)	55.2	88.4	91.6	56.8	89.8	93.5
	Codeforces (Rating)	-	2816	3052	-	2919	3206
	HMMT 2026 Feb (Pass@1)	40.8	91.9	94.8	31.7	94.0	95.2
	IMOAnswerBench (Pass@1)	41.9	85.1	88.4	35.3	88.0	89.8
	Apex (Pass@1)	1.0	19.1	33.0	0.4	27.4	38.3
	Apex Shortlist (Pass@1)	9.3	72.1	85.7	9.2	85.5	90.2

表 7 | 不同规模与模式的 DeepSeek-V4 系列对比。“Non-Think”、“High”和“Max”表示推理强度。

长

MRCR 1M (MMR) 37.576.978.744.783.383.5CorpusQA 1M (ACC) 15.559.360.535.656.562.0

Agentic	Terminal Bench 2.0 (Acc)	49.1	56.6	56.9	59.1	63.3	67.9
	SWE Verified (Resolved)	73.7	78.6	79.0	73.6	79.4	80.6
	SWE Pro (Resolved)	49.1	52.3	52.6	52.1	54.4	55.4
	SWE Multilingual (Resolved)	69.7	70.2	73.3	69.8	74.1	76.2
	BrowseComp (Pass@1)	-	53.5	73.2	-	80.4	83.4
	HLE w/ tools (Pass@1)	-	40.3	45.1	-	44.7	48.2
	MCPAtlas Public (Pass@1)	64.0	67.4	69.0	69.4	74.2	73.6
	GDPval-AA (Elo)	-	-	1395	-	-	1554
	Toolathlon (Pass@1)	40.7	43.5	47.8	46.3	49.0	51.8

推理。DeepSeek-V4-Pro-Max 在各项推理基准上都超过了此前所有开源模型，并在许多指标上达到最先进闭源模型的水平；而更小的 DeepSeek-V4-Flash-Max 也在代码与数学推理任务上超越了此前最佳开源模型 K2.6-Thinking。同时，DeepSeek-V4-Pro 与 DeepSeek-V4-Flash 在编程竞赛中表现突出。根据我们的评测，它们的性能已可与 GPT-5.4 相比，这是开源模型首次在该任务上追平闭源模型。在 Codeforces 排行榜上，DeepSeek-V4-Pro-Max 目前在人类参赛者中排名第 23。DeepSeek-V4 在形式化数学任务上也在智能体设置和高算力设置下都表现强劲。在智能体设置下，它取得了最先进结果，如图 8 所示，优于 Seed Prover (Chen et al., 2025) 等以往模型。在更高算力的流水线下，其性能进一步提升，超过了 Aristotle (Achim et al., 2025) 等系统，并达到了该设置下已知最佳结果的水平。

智能体。DeepSeek-V4 系列在各项评测中展现出强劲的智能体表现。在代码智能体任务上，DeepSeek-V4-Pro 取得了与 K2.6 和 GLM-5.1 相当的结果，但这些开源模型整体上仍落后于闭源对应模型。DeepSeek-V4-Flash 在编码任务上的表现弱于 DeepSeek-V4-Pro，尤其是在 Terminal Bench 2.0 上。类似趋势也出现在其他智能体评测中。值得注意的是，DeepSeek-V4-Pro 在 MCPAtlas 和 Toolathlon 这两个包含广泛工具与 MCP 服务的评测集上表现良好，这表明我们的模型具有出色的泛化能力，而不仅仅是在内部框架上表现出色。

原文第 40 页

前沿设置Putnam-2025，采用形式化与非形式化相结合的混合推理，并进行大幅算力扩展。

实用设置Putnam-200 Pass@8，使用最少工具并限制采样规模。



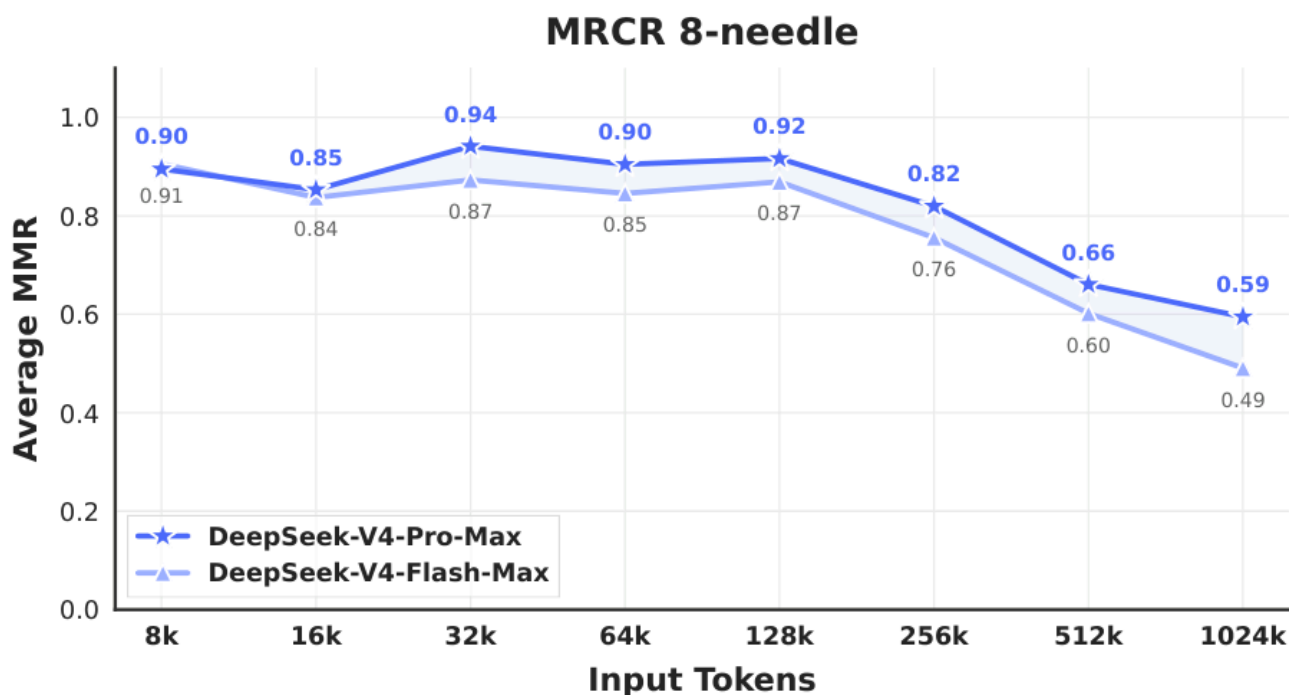
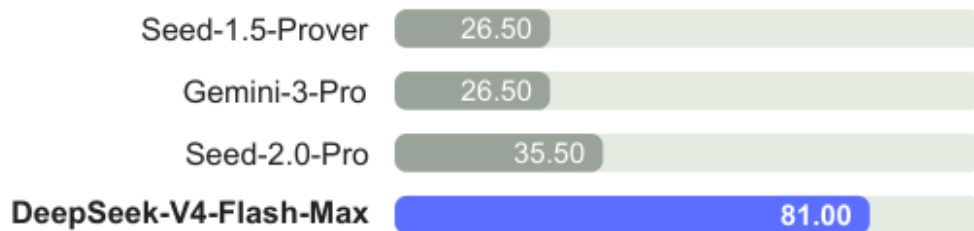
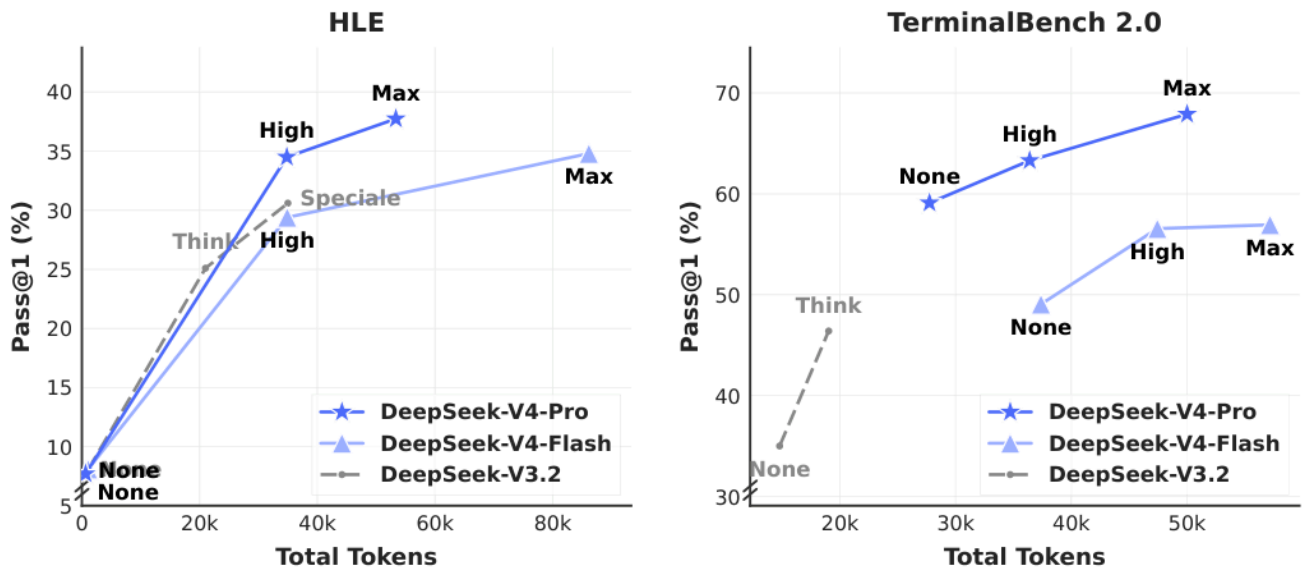


图 8 | 实用设置与前沿设置下的形式化推理。左图：Putnam-200 Pass@8 按照 Seed-Prover 提出的设置，对 PutnamBench (Tsoukalas et al., 2024) 中的一个固定随机子集进行评测；所有模型都在同一题集上测试。我们遵循 Seed-Prover 协议，但将专有搜索工具替换为开源 LeanExplore (Asher, 2025)，从而形成一个只使用最少智能体工具并限制采样规模的轻量级设置。右图：Putnam-2025 探测在扩展后的形式化-非形式化混合设置下数学推理的前沿，其中非形式化推理与形式验证相结合，以暴露漏洞并提升严谨性；DeepSeek-V4 达到了证明完美的 120/120。

1M-Token 上下文。DeepSeek-V4-Pro 在衡量上下文内检索能力的 MRCR 任务上优于 Gemini-3.1-Pro，但仍落后于 Claude Opus 4.6。如图 9 所示，在 128K 上下文窗口内，检索性能始终高度稳定。虽然超过 128K 后性能开始出现下降，但在 1M Token 时，模型的检索能力相较于闭源和开源对手依然非常强。与 MRCR 不同，CorpusQA 更接近真实场景。评测结果同样表明，DeepSeek-V4-Pro 优于 Gemini-3.1-Pro。

推理强度。如表 7 所示，Max 模式采用更长的上下文，并在 RL 中使用更低(length penalty)，因此最具挑战性的任务上优于 High 模式。图 10 展示了 DeepSeek-V4-Pro、DeepSeek-V4-Flash 与 DeepSeek-V3.2 在代表性推理和智能体任务上的性能与成本对比。通过扩展测试时算力，DeepSeek-V4 系列相较前代实现了显著提升。此外，在 HLE 等推理任务上，DeepSeek-V4-Pro 展现出比

图 9 | DeepSeek-V4 系列在 MRCR 任务上的表现。



DeepSeek-V3.2 更高的 Token 效率。

5.4. 真实世界任务表现

标准化基准往往难以捕捉多样化真实任务中的复杂性，因此测试结果与实际用户体验之间会存在鸿沟。为弥补这一点，我们开发了专有的内部指标，更重视真实使用模式而非传统基准。这种方法确保我们的优化能够转化为切实收益。我们的评测框架专门针对 DeepSeek API 和 Chatbot 的主要使用场景，使模型性能与实际需求保持一致。

5.4.1. 中文写作

中文写作是 DeepSeek 的主要使用场景之一。我们对功能性写作和创意写作进行了严格评测。表 12 展示了 DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在功能性写作任务上的两两对比。这些任务由常见的日常写作请求构成，提示通常简短直接。我们选择 Gemini-3.1-Pro 作为基线，因为在我们的评测中，它是外部模型里中文写作表现最好的模型。结果表明，DeepSeek-V4-Pro 以 62.7% 对 34.1% 的总体胜率优于基线；主要原因在于，Gemini 在中文写作场景中有时会让其固有文风偏好压过用户的明确要求。

表 13 展示了创意写作对比，沿两个维度进行评估：指令遵循和写作质量。与 Gemini-3.1-Pro 相比，DeepSeek-V4-Pro 在指令遵循上的胜率为 60.0%，在写作质量上的胜率为 77.5%，说明其在指令遵循上略有提升，在写作质量上则有显著优势。尽管在总体用户案例分析中 DeepSeek-V4-Pro 表现更优，但当评测仅限于最具挑战性的提示——尤其是涉及高复杂度约束或多轮场景的提示——时，Claude Opus 4.5 仍然相对 DeepSeek-V4-Pro 保持性能优势。如表 14 所示，Claude Opus 4.5 的胜率为 52.0%，而 DeepSeek-V4-Pro 为 45.9%。

5.4.2. 搜索

搜索增强问答是 DeepSeek 聊天机器人的核心能力之一。在 DeepSeek 网页端和 App 中，“non-think”模式使用检索增强搜索（RAG），而“thinking”模式使用智能体搜索。

检索增强搜索。我们对 DeepSeek-V4-Pro 与 DeepSeek-V3.2 在客观问答和主观问答两类任务上进行了成对评测。如表 11 所示，DeepSeek-V4-Pro 显著优于 DeepSeek-V3.2，并在两类任务上都表现出稳定优势。其中提升最明显的是单值搜索以及规划与策略任务，这表明 DeepSeek-V4-Pro 更擅长从检索上下文中定位精确事实答案，并综合生成结构化方案。不过，DeepSeek-V3.2 在比较类与推荐类任务上仍具有相对竞争力，这说明 DeepSeek-V4-Pro 在需要基于搜索结果进行平衡、多视角推理的场景中仍有提升空间。

智能体搜索。不同于标准 RAG，智能体搜索使模型能够针对每个查询迭代调用搜索和抓取工具，从而显著提升整体搜索表现。对于 DeepSeek-Chat 中的 thinking 模式，我们优化了智能体搜索功能，以在预先设定的“思考预算”内最大化回答准确率。如表 9 所示，智能体搜索始终优于 RAG，尤其是在复杂任务上。此外，它的成本仍然非常高效，智能体搜索仅比标准 RAG 略贵一些（见表 10）。

5.4.3. 白领任务

为了严格评估模型在复杂企业生产力场景中的实用性，我们构建了一套包含 30 个高级中文专业任务的综合评测集。这些 workflow 刻意覆盖高层次认知需求，包括深入信息分析、完整文档生成以及细致文档编辑，横跨 13 个关键行业（如金融、教育、法律和科技）。评测在一个配备基础工具（包括 Bash 和网页搜索）的内部智能体 harness 中进行。

鉴于这些任务具有开放性，自动化指标通常难以捕捉高质量回答中的细微差别。因此，我们通过人工评估，将 DeepSeek-V4-Pro-Max 与 Opus-4.6-Max 的表现进行比较。标注人员在盲测条件下从四个维度评估模型输出：

- 任务完成度：核心问题是否得到成功解决。
- 指令遵循：是否遵守具体限制与要求。
- 内容质量：事实准确性、逻辑连贯性与专业语气。
- 格式美观度：版式可读性与视觉呈现。

如图 11 所示，DeepSeek-V4-Pro-Max 在多样化中文白领任务上优于 Opus-4.6-Max，取得了令人印象深刻的 63% 不败率，并在分析、生成和编辑任务上都展现出持续优势。图 12 中给出的详细维度得分表明，该模型的主要优势体现在任务完成度与

内容质量。具体而言，DeepSeek-V4-Pro-Max 会主动预判用户的隐含意图，频繁提供补充见解和自我验证步骤。它在长篇生成方面也表现出色，能够输出深入、连贯的叙述，而不是依赖 Opus-4.6-Max 常产生的过于简单的要点式内容。此外，该模型严格遵循正式专业规范，例如标

准化的中文层级编号。然而，在指令遵循方面，它偶尔会忽略特定格式约束，略逊于 Opus。再者，该模型不太擅长将大量文本输入压缩为简洁摘要。最后，在演示文稿整体视觉设计方面，其格式美观度仍有较大提升空间。图 13、14 和 15 展示了若干测试案例；由于部分输出篇幅很长，这里仅展示了部分页面。

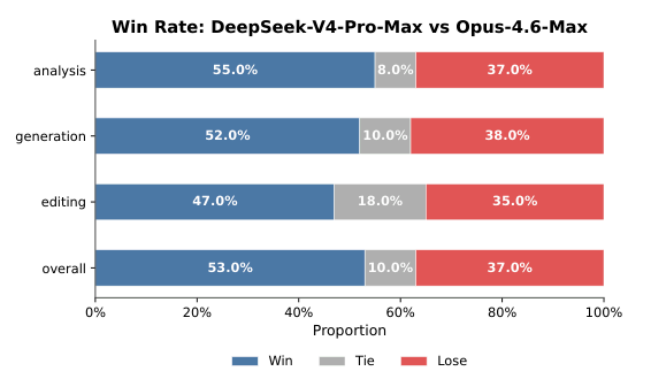


Figure 11 | Win-rate comparison across analysis, generation, editing tasks, and the overall performance.

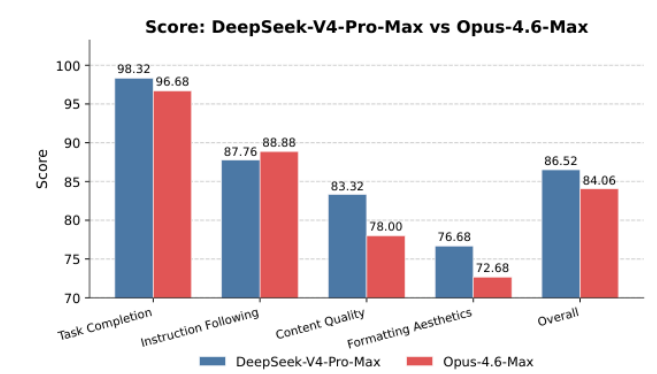


Figure 12 | Detailed dimension scores including Task Completion, Content Quality, Formatting Aesthetics, and Instruction Following.



图 13 | 一个任务输出示例：需要起草某热门奶茶品牌与北京地铁的联合营销提案。

5.4.4. 代码智能体

为了对代码智能体能力进行基准测试，我们从真实的内部研发 workflows 中整理任务。我们从 50 多位内部工程师那里收集了约 200 个具有挑战性的任务，涵盖功能开发、缺陷修复、重构和诊断，涉及 PyTorch、CUDA、Rust 和 C++ 等多种技术栈。每个任务都附带原始代码仓库、对应执行环境以及人工标注的评分标准；经过严格质量筛选后，最终保留 30 个任务作为评测集。如表 8 所示，DeepSeek-V4-Pro 明显优于 Claude Sonnet 4.5，并接近 Claude Opus 4.5 的水平。

模 型	Haiku	4.5	Sonnet	4.5	DeepSeek-V4-Pro-Max	Opus	4.5	Opus	4.5	Thinking	Opus
	4.6	Thinking									

通过率 (%) 134767707380

在一项面向 DeepSeek 开发者和研究人员 ($N = 85$) 的调查中——这些人都在日常工作中使用过 DeepSeek-V4-Pro 进行智能体编码——我们询问：与其他前沿模型相比，DeepSeek-V4-Pro 是否已经可以作为他们默认且首选的代码模型。结果显示，52% 的人回答“是”，39% 的人倾向于“是”，不到 9% 的人回答“否”。受访者认为 DeepSeek-V4-Pro 在大多数任务上都能给出令人满意的结果，但也指出其会出现琐碎错误、误解含糊提示以及偶尔过度思考等问题。

6. 结论、局限性与未来方向

在本文中，我们展示了 DeepSeek-V4 系列的预览版本，其目标是打造突破超长上下文处理效率瓶颈的下一代大语言模型。通过结合融合 CSA 与 HCA 的混合注意力架构，DeepSeek-V4 系列在长序列效率上实现了显著跃升。这些架构创新连同大规模基础设施优化，使模型能够高效原生支持百万 Token 上下文，并为未来测试时扩展、长时程任务以及在线学习等新兴范式奠定了必要基础。评测结果表明，DeepSeek-V4-Pro-Max 作为 DeepSeek-V4-Pro 的最高推理强度模式，重新定义了开源模型的最先进水平。它在知识基准上大幅超过先前的开源模型，在推理性能上逼近前沿专有模型，并具备具有竞争力的智能体能力。同时，DeepSeek-V4-Flash-Max 在保持高度成本效率架构的同时，达到了可与领先闭源模型相比的推理性能。我们相信，DeepSeek-V4 系列为开源模型开启了百万长度上下文的新纪元，并为更高的效率、规模和智能铺平了道路。

为追求极致的长上下文效率，DeepSeek-V4 系列采用了大胆的架构设计。为了尽量降低风险，我们保留了许多经过初步验证的组件和技巧；它们虽然有效，但也使整体架构相对复杂。未来迭代中，我们将开展更全面、更有原则的研究，提炼出最本质的设计，使架构在不牺牲性能的前提下更加优雅。同时，尽管 Anticipatory Routing 和 SwiGLU Clamping 已被证明能够有效缓解训练不稳定性，但其底层原理仍缺乏充分理解。我们将积极研究训练稳定性的基础问题，并加强内部指标监控，以期形成更有原则、也更具预测性的稳定大规模训练方法。

表 8 | 研发代码基准对比（其中外部模型仅用于评测目的）。

此外，除了 MoE 和稀疏注意力架构之外，我们还将主动探索新的模型稀疏性维度——例如更稀疏的嵌入模块（Cheng et al., 2026）——以在不损害能力的前提下进一步提升计算与内存效率。我们也将持续研究低时延架构和系统技术，使长上下文部署与交互更加灵敏高效。此外，我们认识到长时程、多轮智能体任务的重要性和实际价值，并将继续沿这一方向迭代探索。我们也在推进将多模态能力整合进模型。最后，我们致力于开发更好的数据整理与合成策略，以在越来越广泛的场景和任务中持续提升模型的智能性、鲁棒性与实用性。

参考文献

AA. GDPval-AA 排行榜, 2025。URL <https://artificialanalysis.ai/methodology/intelligence-benchmarking#gdpval-aa>。

T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, et al. Aristotle: IMO 水平的自动定理证明。arXiv 预印本 arXiv:2510.01346, 2025。

A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: 面向无服务器应用的轻量级虚拟化。发表于第 17 届 USENIX 网络系统设计与实现会议（NSDI’ 20）论文集，第 419–434 页，美国，2020。USENIX Association. ISBN 9781939133137。

O. J. Aimuyo, B. Oh, and R. Singh. FlashMoE: 在单一内核中实现快速分布式 MoE。发表于《神经信息处理系统进展》

, 2025。URL <https://neurips.cc/virtual/2025/poster/119124>。

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. GQA: 从多头检查点训练广义多查询 Transformer 模型。arXiv 预印本 arXiv:2305.13245, 2023。

J. Asher. LeanExplore: Lean 4 声明搜索引擎, 2025。URL <https://arxiv.org/abs/2506.11085>。

Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, H. Gao, P. Gao, T. Gao, X. Gu, L. Guan, H. Guo, J. Guo, H. Hu, X. Hao, T. He, W. He, W. He, C. Hong, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, L. Lu, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S. Pan, Y. Peng, R. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, X. Sun, F. Sung, H. Tang, J. Tao, Q. Teng, C. Wang, D. Wang, F.

Wang, and H. Wang. Kimi K2: 开放式智能体智能。CoRR, abs/2507.20534, 2025a。URL <https://doi.org/10.48550/arXiv.2507.20534>。

Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. LongBench v2: 迈向对真实长上下文多任务更深入的理解与推理。发表于第 63 届计算语言学协会年会论文集 (第 1 卷: 长论文), 第 3639–3664 页, 2025b。

原文第 46 页

M. Balunović, J. Dekoninck, I. Petrov, N. Jovanović, and M. Vechev. MathArena: 在未污染数学竞赛上评测大语言模型。发表于神经信息处理系统数据集与基准专题论文集, 2025。

C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. MCP-Atlas: 基于真实 MCP 服务器的大规模工具使用能力基准。arXiv 预印本 arXiv:2602.00933, 2026。

F. Bellard.QEMU: 一种快速且可移植的动态翻译器。发表于 USENIX 年度技术会议论文集, ATEC '05, 第 41 页, 美国, 2005。USENIX Association。

I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio. 利用强化学习进行神经组合优化, 2017。URL <https://openreview.net/forum?id=rJY3vK9eg>。

J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-Prover 1.5: 通过从经验中学习掌握本科水平定理证明, 2025。URL <https://arxiv.org/abs/2512.17260>。

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillett, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. 评估在代码上训练的大语言模型。CoRR, abs/2107.03374, 2021。URL <https://arxiv.org/abs/2107.03374>。

T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: 面向深度学习的自动化端到端优化编译器。发表于第 13 届 USENIX 操作系统设计与实现研讨会 (OSDI 18), 第 578–594 页, 美国加州卡

尔 斯 巴 德 , 2018 。 USENIX Association 。 ISBN 978-1-939133-08-3 。 URL <https://www.usenix.org/conference/osdi18/presentation/chen>。

A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S.Tomar, L. Haas, et al. Facts 排行榜：大语言模型事实性的综合基准。arXiv 预印本 arXiv:2512.10791, 2025。

X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. 通过可扩展查找实现条件记忆：大语言模型稀疏性的新维度。CoRR, abs/2601.07372, 2026 。 doi: 10.48550/ARXIV.2601.07372 。 URL <https://doi.org/10.48550/arXiv.2601.07372>。

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. 训练验证器以求解数学应用题。arXiv 预印本 arXiv:2110.14168, 2021。

D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K.Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. DeepSeekMoE：迈向混合专家语言模型中的终极专家专业化。CoRR, abs/2401.06066, 2024 。 URL <https://doi.org/10.48550/arXiv.2401.06066>。

原文第 47 页

T. Dao, D. Haziza, F. Massa, and G. Sizov. 面向长上下文推理的 Flash-Decoding, 2023。URL <https://pytorch.org/blog/flash-decoding/>。

L. De Moura and N. Bjørner.Z3：高效的 SMT 求解器。发表于 Theory and Practice of Software 第 14 届工具与算法构造和系统分析国际会议 (TACAS' 08/ETAPS' 08) 论文集, 第 337–340 页, 柏林/海德堡, 2008。Springer-Verlag。ISBN 3540787992。

DeepSeek-AI. DeepSeek-Coder-V2：在代码智能上打破闭源模型壁垒。CoRR, abs/2406.11931, 2024。URL <https://doi.org/10.48550/arXiv.2406.11931>。

DeepSeek-AI. DeepSeek-V3 技术报告。CoRR, abs/2412.19437, 2024 。 URL <https://doi.org/10.48550/arXiv.2412.19437>。

DeepSeek-AI. DeepSeek-V2：一种强大、经济且高效的混合专家语言模型。CoRR, abs/2405.04434, 2024。URL <https://doi.org/10.48550/arXiv.2405.04434>。

DeepSeek-AI. Fire-Flyer 文件系统, 2025。URL <https://github.com/deepseek-ai/3FS>。

DeepSeek-AI. DeepSeek-R1：通过强化学习激励大语言模型推理。Nat., 645(8081):633–638, 2025。URL <https://doi.org/10.1038/s41586-025-09422-z>。

DeepSeek-AI. DeepSeek-V3.2：推动开放大语言模型的前沿，2025。URL <https://arxiv.org/abs/2512.02556>。

X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. SWE-Bench Pro：AI 智能体能否解决长时程软件工程任务？，2025。URL <https://arxiv.org/abs/2509.16941>。

H. Ding, Z. Wang, G. Paolini, V. Kumar, A. Deoras, D. Roth, and S. Soatto. 更少截断可改善语言建模。arXiv 预印本 arXiv:2404.10830, 2024。

X. Dong, Y. Fu, S. Diao, W. Byeon, Z. CHEN, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V. keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, and P. Molchanov. Hymba：面向小型语言模型的混合头架构。发表于第十三届国际学习表征会议，2025。URL <https://openreview.net/forum?id=A1ztozypga>。

X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. SuperGPQA：将大语言模型评测扩展至 285 个研究生学科。arXiv 预印本 arXiv:2502.14739, 2025。

D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP：一种要求对段落进行离散推理的阅读理解基准。发表于 2019 年北美计算语言学协会人类语言技术会议 (NAACL-HLT 2019) 论文集第 1 卷 (长短论文)，第 2368–2378 页。Association for Computational Linguistics,

X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. EROFS：一种面向资源受限设备、对压缩友好的只读文件系统。发表于 2019 年 USENIX 年度技术会议论文集 (USENIX ATC '19)，第 149–162 页，美国，2019。USENIX Association. ISBN 9781939133038。

原文第 48 页

A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P. Minervini. 我们已经做完 MMLU 了吗？CoRR, abs/2406.04127, 2024。URL <https://doi.org/10.48550/arXiv.2406.04127>。

F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. 通过多 Token 预测实现更好且更快的大语言模型。发表于第 41 届国际机器学习会议 (ICML 2024)，奥地利维也纳，

2024 年 7 月 21–27 日。OpenReview.net, 2024。URL <https://openreview.net/forum?id=pEWAcejiU2>。

L. Haas, G. Yona, G. D' Antonio, S. Goldshtein, and D. Das. SimpleQA Verified: 用于衡量参数知识的可靠事实性基准。arXiv 预印本 arXiv:2509.07968, 2025。

Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese SimpleQA: 面向大语言模型的中文事实性评测。arXiv 预印本 arXiv:2411.07140, 2024。

D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. 衡量大规模多任务语言理解。arXiv 预印本 arXiv:2009.03300, 2020。

D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. 通过 MATH 数据集衡量数学问题求解能力。arXiv 预印本 arXiv:2103.03874, 2021。

Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: 面向基础模型的多层级、多学科中文评测套件。arXiv 预印本 arXiv:2305.08322, 2023。

D. Hupkes and N. Bogoychev. MultiLoKo: 覆盖 31 种语言的大语言模型多语种本地知识基准。CoRR, abs/2504.10356, 2025。doi: 10.48550/ARXIV.2504.10356。URL <https://doi.org/10.48550/arXiv.2504.10356>。

B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. 面向高效纯整数算术推理的神经网络量化与训练。发表于 IEEE 计算机视觉与模式识别会议 (CVPR), 2018 年 6 月。

N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: 面向代码的大语言模型整体且无污染评测。arXiv 预印本 arXiv:2403.07974, 2024。

K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: 神经网络隐藏层优化器。Cited on, page 10, 2024。

M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: 大规模远程监督阅读理解挑战数据集。发表于第 55 届计算语言学协会年会论文集 (第 1 卷: 长论文), 第 1601–1611 页, 加拿大温哥华, 2017 年 7 月。Association for Computational Linguistics。doi: 10.18653/v1/P17-1147。URL <https://aclanthology.org/P17-1147>。

H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: 用于敏捷与弹性应用部署的块级镜像服务。发表于 2020 年 USENIX 年度技术会议 (USENIX ATC 20), 第 727–740 页。USENIX

Association , 2020 年 7 月 。 ISBN 978-1-939133-14-4 。 URL <https://www.usenix.org/conference/atc20/presentation/li-huiba>。

原文第 49 页

H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: 衡量中文中的大规模多任务语言理解。arXiv 预印本 arXiv:2306.09212, 2023。

J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. Tool Decathlon : 面向多样、真实且长时程任务执行的语言智能体基准。arXiv 预印本 arXiv:2510.25726, 2025。

Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: 投机采样要求重新思考特征不确定性。发表于第 41 届国际机器学习会议 (ICML 2024) , 奥地利维也纳, 2024 年 7 月 21–27 日。OpenReview.net, 2024。URL <https://openreview.net/forum?id=1NdN7eXyb4>。

J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon 可扩展用于大语言模型训练。CoRR, abs/2502.16982, 2025。URL <https://doi.org/10.48550/arXiv.2502.16982>。

I. Loshchilov and F. Hutter.解耦式权重衰减正则化。arXiv 预印本 arXiv:1711.05101, 2017。

K. Lu and T. M. Lab. On-policy distillation : 在线策略蒸馏。Thinking Machines Lab: Connectionism, 2025 。 doi: 10.64434/tml.20251026 。 <https://thinkingmachines.ai/blog/on-policy-distillation>。

Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan, and F. Huang. CorpusQA: 用于语料级分析与推理的千万 Token 基准。arXiv 预印本 arXiv:2601.14952, 2026。

T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. 迈向稳健的数学推理。发表于 2025 年自然语言处理经验方法会议论文集, 2025。URL <https://aclanthology.org/2025.emnlp-main.1794/>。

M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-Bench: 在命令行界面中的困难真实任务上评测智能体。arXiv 预印本 arXiv:2601.11868, 2026。

MiniMax. 认识 MiniMax-M2, 2025。URL <https://github.com/MiniMax-AI/MiniMax-M2>。

L. d. Moura and S. Ullrich. **Lean 4 定理证明器与编程语言**。发表于国际自动推理会议，第 625–635 页。Springer, 2021。

Y. Nesterov. **以 $O(1/k^2)$ 收敛率求解凸规划问题的方法**。

《Soviet Mathematics Doklady》，27:372–376, 1983。

NVIDIA Corporation. cuBLAS 文档，2024。URL <https://docs.nvidia.com/cuda/cublas/>。Version 12.4。访问时间：2024-09-16。

OpenAI. 多语种大规模多任务语言理解（MMMLU），2024a。URL <https://huggingface.co/datasets/openai/MMMLU>。

OpenAI. OpenAI MRCR：长上下文多针草堆基准，2024b。URL <https://huggingface.co/datasets/openai/mrcr>。

原文第 50 页

OpenAI. 让大语言模型学会推理，2024c。URL <https://openai.com/index/learning-to-reason-with-llms>。

OpenAI. 介绍 SimpleQA，2024d。URL <https://openai.com/index/introducing-simpleqa/>。

OpenAI. 介绍 SWE-bench Verified：我们发布了 SWE-bench 的人工验证子集，2024e。URL <https://openai.com/index/introducing-swe-bench-verified/>。

OpenAI. gpt-oss-120b 与 gpt-oss-20b 模型卡。CoRR, abs/2508.10925, 2025。doi: 10.48550/ARXIV.2508.10925。URL <https://doi.org/10.48550/arXiv.2508.10925>。

M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-K：面向 GPU 稠密矩阵乘法的以工作为中心的并行分解。发表于第 28 届 ACM SIGPLAN 并程序设计与实践年会论文集，第 429–431 页，2023。

T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubeh, P. Thacker, L. Fauconnet, et al. GDPval：在真实世界中具有经济价值的任务上评估 AI 模型表现。arXiv 预印本 arXiv:2510.04374, 2025。

L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. 《Humanity’s Last Exam》。arXiv 预印本 arXiv:2501.14249, 2025。

W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, and M. Zhou. ProphetNet：用于序列到序列预训练的未来 n-gram 预测。发表于 ACL Findings: EMNLP 2020，第 2401–2410

页, 2020。 Association for Computational Linguistics。 URL <https://doi.org/10.18653/v1/2020.findings-emnlp.217>。

Qwen. Qwen3 技术报告。 CoRR, abs/2505.09388, 2025。 doi: 10.48550/ARXIV.2505.09388。 URL <https://doi.org/10.48550/arXiv.2505.09388>。

S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. ZeRO: 面向万亿参数模型训练的内存优化。发表于 SC20: 高性能计算、网络、存储与分析国际会议, 第 1–16 页。 IEEE, 2020。

J. K. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. Torch.fx: 面向 Python 深度学习的实用程序捕获与变换, 2022。 URL <https://arxiv.org/abs/2112.08429>。

D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: 研究生水平、Google 难以检索答案的问答基准。 arXiv 预印本 arXiv:2311.12022, 2023。

G. T. M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ram' e, J. Ferret, P. Liu, P. D. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P. Sta'nczyk, S. Girgin, N. Momchev, M. Hoff-man, S. Thakoor, J.-B. Grill, B. Neyshabur, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson, B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. Choquette-Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozińska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshev, F. Visin, G. Rasskin, G. Wei, G. Cameron,

G. Martins, H. Hashemi, H. Klimczak-Plucińska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjoesund, L. Usui, L. Sifre, L. Heuermann, L. cia Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Gerner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P. Botarda, P. Barnes, P. Barham, P. Michel, P. chong Jin, P. Georgiev, P. Culliton, P. Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S.

bastian Krause, S. Dai, S. Garg, S. Sheth, S. Ronstrom, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Kociský, T. Doshi, V. Jain, V. Yadav, V. Meshram, V. Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V. Cotruta, P. Kirk, A. Rao, M. Giang, L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, and A. Andreev. Gemma 2: 在实用规模上改进开放语言模型。arXiv 预印本 arXiv:2408.00118, 2024。

S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. 用于大型稀疏模型的哈希层。发表于 NeurIPS 2021, 第 17555–17566 页。URL <https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737e0326ea59846a83e076-Abstract.html>。

B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. 面向深度学习的微缩放数据格式, 2023。

K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. WinoGrande: 大规模对抗式 Winograd Schema 挑战, 2019。

Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. DeepSeekMath-V2: 迈向可自验证的数学推理, 2025。URL <https://arxiv.org/abs/2511.22570>。

N. Shazeer. 快速 Transformer 解码: 一个写入头就足够了。CoRR, abs/1911.02150, 2019。URL <http://arxiv.org/abs/1911.02150>。

N. Shazeer. GLU 变体可改进 Transformer。arXiv 预印本 arXiv:2002.05202, 2020。

F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. 语言模型是多语言链式思维推理者。发表于第十一届国际学习表征会议 (ICLR 2023), 卢旺达基加利, 2023 年 5 月 1–5 日。OpenReview.net, 2023。URL <https://openreview.net/forum?id=fR3wGCK-IXp>。

J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: 带有旋转位置嵌入的增强型 Transformer。Neurocomputing, 568:127063, 2024。

M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. 具有挑战性的 BIG-Bench 任务, 以及链式思维是否能够解决它们。arXiv 预印本 arXiv:2210.09261, 2022。

G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. PutnamBench: 在 Putnam 数学竞赛上评测神经定理证明器, 2024。URL <https://arxiv.org/abs/2407.11214>。

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention Is All You Need: 《注意力就是你所需要的一切》。发表于《神经信息处理系统进展》, 30, 2017。

L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. 面向混合专家的无辅助损失负载均衡策略。CoRR, abs/2408.15664, 2024a。URL <https://doi.org/10.48550/arXiv.2408.15664>。

L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, et al. TileLang: 在现代神经网络内核中连接可编程性与性能。发表于第十四届国际学习表征会议, 2026。

Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. MMLU-Pro: 一个更稳健且更具挑战性的多任务语言理解基准。CoRR, abs/2406.01574, 2024b。URL <https://doi.org/10.48550/arXiv.2406.01574>。

J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. BrowseComp: 一个简单却具有挑战性的浏览智能体基准。arXiv 预印本 arXiv:2504.12516, 2025。

T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. CMath: 你的语言模型能通过中国小学数学测试吗? , 2023。

G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. 具有 attention sinks 的高效流式语言模型。发表于第十二届国际学习表征会议 (ICLR 2024), 奥地利维也纳, 2024 年 5 月 7-11 日。OpenReview.net, 2024。URL <https://openreview.net/forum?id=NG7sS51zVF>。

Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mHC: 流形约束超连接, 2026。URL <https://arxiv.org/abs/2512.24880>。

L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H.

Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: 中文语言理解评测基准。发表于第 28 届国际计算语言学会议 (COLING 2020) 论文集, 第 4762–4772 页, 2020。doi: 10.18653/V1/2020.COLING-MAIN.419。URL <https://doi.org/10.18653/v1/2020.coling-main.419>。

J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. SWE-Smith: 为软件工程智能体扩展数据, 2025。URL <https://arxiv.org/abs/2504.21798>。

原文第 53 页

R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: 机器真的能续写你的句子吗? 发表于第 57 届计算语言学协会年会 (ACL 2019) 论文集第 1 卷: 长论文, 第 4791–4800 页。Association for Computational Linguistics, 2019。doi: 10.18653/v1/p19-1472。URL <https://doi.org/10.18653/v1/p19-1472>。

C. Zhang, K. Du, S. Liu, W. Kwon, X. Mo, Y. Wang, X. Liu, K. You, Z. Li, M. Long, J. Zhai, J. Gonzalez, and I. Stoica. Jenga: 面向异构场景的大语言模型服务有效内存管理。发表于 ACM SIGOPS 第 31 届操作系统原理研讨会 (SOSP '25) 论文集, 第 446–461 页, 纽约, 2025a。Association for Computing Machinery。ISBN 9798400718700。doi: 10.1145/3731569.3764823。URL <https://doi.org/10.1145/3731569.3764823>。

S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu. Comet: 面向混合专家的细粒度计算-通信重叠。2025b。URL <https://arxiv.org/abs/2502.19811>。

C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. DeepGEMM: 具有细粒度缩放的简洁高效 FP8 GEMM 内核。 <https://github.com/deepseek-ai/DeepGEMM>, 2025。

W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: 以人为中心的基础模型评测基准。CoRR, abs/2304.06364, 2023。doi: 10.48550/arXiv.2304.06364。URL <https://doi.org/10.48550/arXiv.2304.06364>。

D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. 超连接。发表于第十三届国际学习表征会议 (ICLR 2025), 新加坡, 2025 年 4 月 24–28 日。OpenReview.net, 2025。URL <https://openreview.net/forum?id=9FqARW7dwB>。

X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. 如何在不导致模型坍塌的情况下合成文本数据? arXiv 预印本 arXiv:2412.14689, 2024。

T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, and et al. BigCodeBench: 使用多样函数调用和复杂指令评测代码生成。发表于第十三届国际学习表征会议 (ICLR 2025) , 新加坡, 2025 年 4 月 24-28 日。OpenReview.net, 2025。URL <https://openreview.net/forum?id=YrycTjllL0>。

原文第 54 页

附录

A. 作者列表与致谢

A.1. 作者列表

作者按名字首字母顺序排列。带 * 标记的姓名表示这些成员已经离开我们的团队。

研究与工程: Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang*, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chenze Shao, Chong Ruan*, Conner Sun, Damai Dai, Daya Guo*, Dejian Yang, Deli Chen, Donghao Li, Erhang Li, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei*, Haotian Yuan, Haowei Zhang*, Haowen Luo, Haoyu Chen, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J. Yang, J.Q. Zhu, Jia Yu, Jialiang Huang, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran*, Junguang Jiang, Junjie Qiu, Junlong Li*, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang*, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, M.S. Di, M.Y. Xu, Max Mei, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S.H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W. Zhang, Wangding Zeng, Weilin

Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, WenjunGao, Wenkai Yang, Wenlve Huang, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, XiangwenWang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie,Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang,Xu Chen, Xuanyu Wang, Xuecheng Su, Xuheng Lin, Xuwei Fu, Y.C. Yan, Y.Q. Wang*, Y.W. Ma,Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Zhao,Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, YiliangXiong, Ying He, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang,Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo,Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yudian Wang, YuhanWu, Yuhao Meng, Yuheng Zou, YuKun Li, Yunfan Xiong, Yupeng Chen, Yuqian Cao, YuqianWang, Yushun Zhang, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, YuxuanZhou, Yuyang Zhou, Yuzhen Huang, Z.F. Wu, Zehao Wang, Zehua Zhao, Zehui Ren, ZhangliSha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma,Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, ZhuoshuLi, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang*, ZiweiXie, Ziyi Gao, Zizheng Pan, Zongqing Yao.

商务与合规：Chenchen Ling, Chengyu Hou, Dongjie Ji, Fang Wei, Hengqing Zhang,Jia Luo, Jia Song, Jialu Cai, Jian Liang, Jiangting Zhou, Jieyu Yang, Jin Chen, Jingzi Zhou, JunminZheng, Leyi Xia, Linyan Zhu, Miaojun Wang, Mingming Li, Minmin Han, Ning Wang, Panpan

原文第 55 页

Wang, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, W.L. Xiao, Wei An, WenqingHou, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Xinyu Zhang, Xueyin Chen, Yao Xu, YiShao, Yiling Ma, Ying Tang, Yuehan Yang, Yuer Xu, Yukun Zha, Yuping Lin, Yuting Yan, ZekaiZhang, Zhe Ju, Zheren Gao, Zhongyu Wu, Zihua Qu, Ziyi Wan.

A.2. 致谢

我们感谢 Dolly Deng 以及其他测试人员，就 DeepSeek-V4 系列模型的能力提出的宝贵建议与反馈。

B. 评测细节

难度类别#智能体胜RAG 胜平局智能体%RAG%平局%

简单 客观问答（客观问答） 196110434356.121.921.9 主观问答（主观问答） 321198566761.717.420.9

困难 客观问答（客观问答） 168102333360.719.619.6 主观问答（主观问答） 184126273168.514.716.8

总计（总计） 86953615917461.718.320.0

版本工具调用次数Prefill（tokens） 输出（tokens）

V4 智能体搜索16.2136491526V4 检索增强搜索—104531308

内部评估（内部综合评估）

类别子类别#V4 胜V3.2 胜平局V4%V3.2%平局%

单值搜索（单值信息查找） 9536104937.910.551.6 实体搜索（实体信息查找） 992476824.27.168.7枚举型搜索（枚举型信息查找） 951986820.08.471.6

客观问答（客观问答）

小计（小计） 289792518527.38.764.0

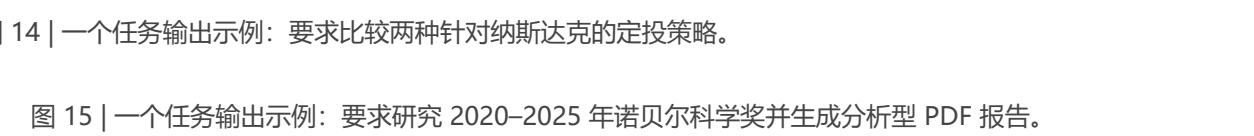
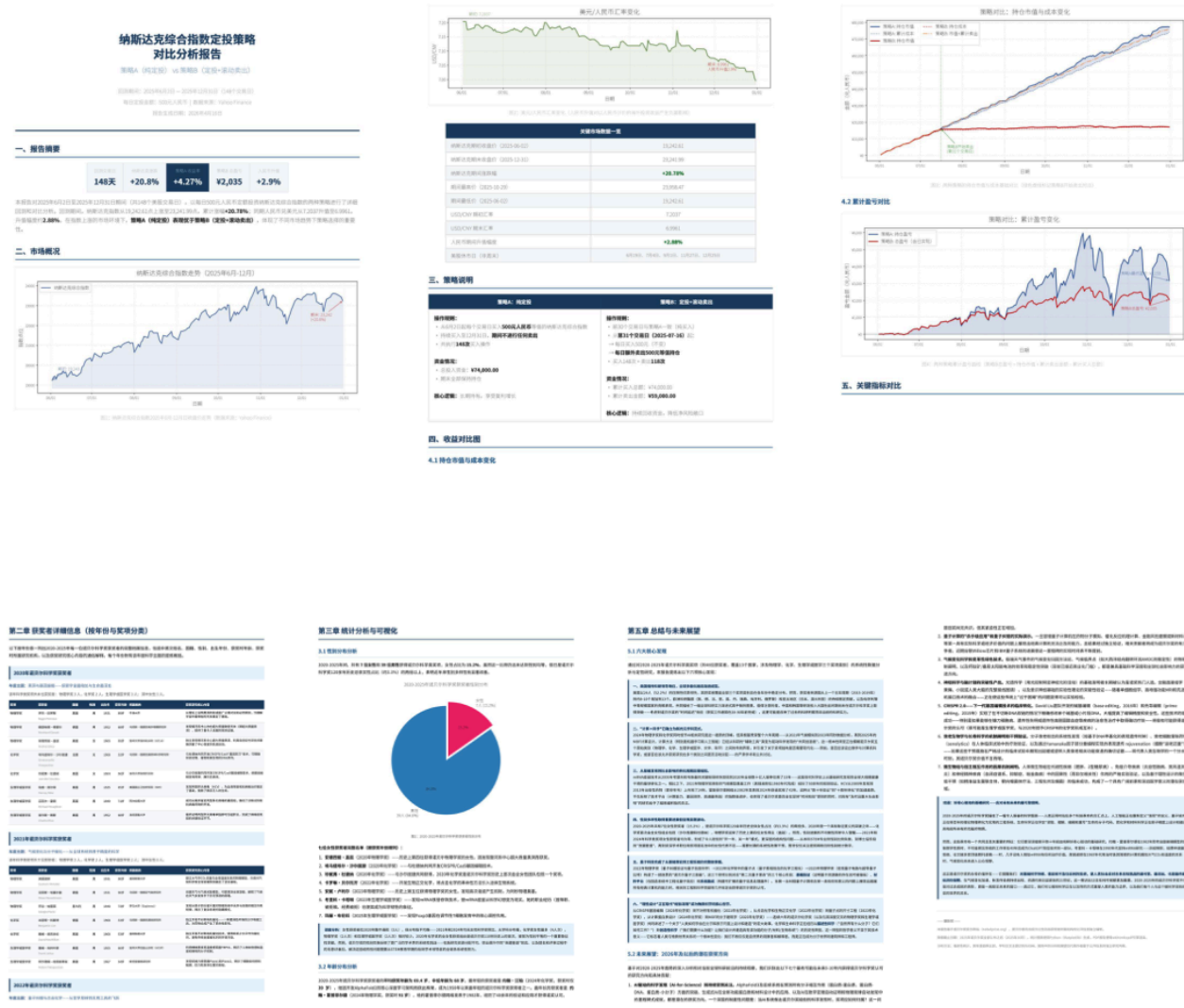
Causal Analysis (原因分析)	100	28	5	67	28.0	5.0	67.0
Comparison (对比)	96	28	20	48	29.2	20.8	50.0
Advice Seeking (寻求建议)	92	23	8	61	25.0	8.7	66.3
Recommendation (推荐)	95	26	19	50	27.4	20.0	52.6
Planning & Strategy (攻略计划)	92	32	11	49	34.8	12.0	53.3
Opinion & Evaluation (评价看法)	96	30	8	58	31.2	8.3	60.4
Trend Analysis (趋势分析)	96	23	3	70	24.0	3.1	72.9

表 11 | DeepSeek-V4-Pro 与 DeepSeek-V3.2 在搜索问答任务上的对比评测。

主观问答（主观问答）

小计（小计） 6671907440328.511.160.4

总计（总计） 9562699958828.110.461.5



内部评估（内部综合评估）

类别子类别#DS 胜Gem 胜平局DS%Gem%平局%

Report (报告)	527	350	162	15	66.41	30.74	2.85
Proposal (方案策划)	291	181	103	7	62.20	35.40	2.41
Education (教育培训)	159	100	56	3	62.89	35.22	1.89
Email & Letter (邮件书信)	146	107	37	2	73.29	25.34	1.37
Notice (通知公告)	72	43	24	5	59.72	33.33	6.94
Professional (专业文本)	63	34	27	2	53.97	42.86	3.17
Recruitment (招聘求职)	42	27	15	0	64.29	35.71	0.00
Technical (技术文本)	29	22	7	0	75.86	24.14	0.00
Review (介绍评价)	20	15	5	0	75.00	25.00	0.00

表 12 | DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在中文功能性写作上的对比分析。

商务写作（办公文本）

小计（小计） 13498794363465.1632.322.52

Social Media (社交媒体文案)	267	156	101	10	58.43	37.83	3.75
Ad Copy (广告商品文案)	214	109	98	7	50.93	45.79	3.27
Long-form Content (内容平台长文)	99	71	25	3	71.72	25.25	3.03
News Report (新闻报道)	51	27	22	2	52.94	43.14	3.92
Advertorial (营销软文)	17	12	4	1	70.59	23.53	5.88
Headline (标题)	11	7	4	0	63.64	36.36	0.00
Narration Script (口播文案)	4	2	1	1	50.00	25.00	25.00
Comment (评论)	3	2	1	0	66.67	33.33	0.00

媒体写作（媒体文本）

小计（小计） 6663862562457.9638.443.60

Congratulatory (祝贺文本)	101	54	41	6	53.47	40.59	5.94
Communication (沟通回复)	100	71	26	3	71.00	26.00	3.00
Reflection (心得感想)	90	68	17	5	75.56	18.89	5.56
Review (介绍评价)	55	44	9	2	80.00	16.36	3.64
Comment (评论)	44	34	8	2	77.27	18.18	4.55

日常写作（生活文本）

小计（小计） 3902711011869.4925.904.62

Speech (发言稿)	226	135	85	6	59.73	37.61	2.65
Narration Script (口播文案)	51	25	23	3	49.02	45.10	5.88
Sales Script (话术)	31	22	6	3	70.97	19.35	9.68
Dialogue (对话文本)	10	4	6	0	40.00	60.00	0.00
Congratulatory (祝贺文本)	1	1	0	0	100.00	0.00	0.00

口语写作 (口头文本)

小计 (小计) 3191871201258.6237.623.76

Administrative Doc (事务文书)	117	60	53	4	51.28	45.30	3.42
Personal Doc (个人文书)	73	45	27	1	61.64	36.99	1.37
Government Doc (行政公文)	34	19	14	1	55.88	41.18	2.94
Speech (发言稿)	3	1	2	0	33.33	66.67	0.00
Essay Writing (申论写作)	3	1	1	1	33.33	33.33	33.33

公文文本 (公文文本)

小计 (小计) 23012697754.7842.173.04

Research Paper (学术论文)	104	67	32	5	64.42	30.77	4.81
Coursework (课程作业)	90	53	35	2	58.89	38.89	2.22
Academic Support (学术辅助)	15	11	3	1	73.33	20.00	6.67
Science Outreach (专业科普)	7	6	1	0	85.71	14.29	0.00

学术写作 (学术文本)

小计 (小计) 21613771863.4332.873.70

总计 (总计) 31701986108110362.6534.103.25

Fiction (小说故事)	836	504	323	5	60.58	38.82	0.60	672	157	3	80.77	18.87	0.36
General Fiction (泛小说故事)	662	368	290	3	55.67	43.87	0.45	467	194	0	70.65	29.35	0.00
Fan Fiction (同人文)	410	253	150	3	62.32	36.95	0.74	338	67	1	83.25	16.50	0.25
General Fan Fic. (泛同人文)	202	111	90	1	54.95	44.55	0.50	161	40	1	79.70	19.80	0.50
Narrative (记叙文)	171	115	54	2	67.25	31.58	1.17	141	30	0	82.46	17.54	0.00
General Prose (泛散文)	124	83	40	1	66.94	32.26	0.81	88	36	0	70.97	29.03	0.00
Prose (散文)	112	74	38	0	66.07	33.93	0.00	92	20	0	82.14	17.86	0.00
Writing Style (文笔)	112	81	31	0	72.32	27.68	0.00	86	26	0	76.79	23.21	0.00
Classical Poetry (古诗文)	48	24	24	0	50.00	50.00	0.00	39	9	0	81.25	18.75	0.00
Modern Poetry (现代诗)	43	23	20	0	53.49	46.51	0.00	32	11	0	74.42	25.58	0.00
Lyrics (歌词)	30	8	22	0	26.67	73.33	0.00	16	14	0	53.33	46.67	0.00
Literary Appreciation (赏析)	27	20	7	0	74.07	25.93	0.00	18	9	0	66.67	33.33	0.00
General Argument. (泛议论文)	24	15	9	0	62.50	37.50	0.00	17	7	0	70.83	29.17	0.00
General Narrative (泛记叙文)	23	11	12	0	47.83	52.17	0.00	15	8	0	65.22	34.78	0.00
General Classical (泛古文诗歌)	9	5	4	0	55.56	44.44	0.00	5	4	0	55.56	44.44	0.00
Creative Writing (创意写作)	6	2	4	0	33.33	66.67	0.00	4	2	0	66.67	33.33	0.00
Argumentative (议论文)	5	5	0	0	100.00	0.00	0.00	5	0	0	100.00	0.00	0.00
General Mod. Poetry (泛现代诗)	2	1	1	0	50.00	50.00	0.00	2	0	0	100.00	0.00	0.00

表 13 | DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在中文创意写作上的对比分析。

总计 (总计) 2837 1703 11191560.0339.440.53 2198634577.4822.350.18

内部评估 (内部综合评估)

类别#DSOpus平局DS%Opus%平局%

复杂指令跟随 (复杂指令跟随) 492326046.9%53.1%0.0% 多轮写作 (多轮写作) 1476776445.6%51.7%2.7%

总计 (总计) 19690102445.9%52.0%2.0%

表 14 | DeepSeek-V4-Pro 与 Claude-Opus-4.5 在复杂指令遵循和多轮写作上的对比。